VECTOR



How to deal with MICROSAR Multi-Core Projects

User Manual

Alexander Zeeb

V2.0.6 | Released

Document Information

History

Author	Date	Version	Remarks
Alexander Zeeb	2018-09-12	1.0.0	Released
Alexander Zeeb	2018-10-29	1.1.0	Updated chapter 4.6, added chapter 4.4.3 and 6.4
Alexander Zeeb	2018-11-12	1.1.1	Supplemented chapter 4.6
Alexander Zeeb	2018-11-22	1.1.2	Added chapter 6.6
Alexander Zeeb	2019-01-25	1.1.3	Removed obsolete reference to AN-ISC-8-1216
Alexander Zeeb	2020-01-24	1.1.4	Updated BSW-Split, Shutdown handling, Dem and FiM initialization
Alexander Zeeb	2020-01-28	1.1.5	Corrected Dem initialization sequence
Alexander Zeeb	2020-01-29	1.1.6	Added Dem_InitMemory() in initialization sequence
Klaus Emmert	2020-07-09	1.1.7	Disclaimer added
Alexander Zeeb	2020-09-03	1.1.8	Added chapter 7.4. Reworked chapter 7.3.8
Alexander Zeeb	2021-04-15	1.1.10	Added chapter
Alexander Zeeb	2021-06-11	1.2.0	Added Multi-Instance Features
Alexander Zeeb	2022-03-07	1.3.0	Rework for MICROSAR R27
Alexander Zeeb	2022-10-17	2.0.0	Rework for MICROSAR R28
Alexander Zeeb	2022-12-23	2.0.1	Added chapter 3.2
Alexander Zeeb	2022-01-05	2.0.2	Corrected chapter 5.18
Alexander Zeeb	2023-01-16	2.0.3	Minor fixes
Alexander Zeeb	2023-01-30	2.0.4	Removed reference to SoAd solving action in chapter 6.3.3
Alexander Zeeb	2023-02-14	2.0.5	Correct TryToGetSpinlock API in chapter 6.3.15
Alexander Zeeb	2023-02-17	2.0.6	Supplemented chapter 3.2

Reference Documents

No.	Source	Title	Version
[1]	Vector	Technical Reference MICROSAR Diagnostic Event Manager (Dem)	22.05.00
[2]	Vector	AN-ISC-8-1230 Hardware Specific Compare-and-Swap	1.0.0
[3]	Vector	Technical Reference MICROSAR Diagnostic Event Manager (Dem)	19.1.0
[4]	Vector	Technical Reference MICROSAR OS	2.35.00
[5]	Vector	Technical Reference MICROSAR FiM	9.00.00
[6]	Vector	Technical Reference MICROSAR Basic Software Mode Manager	15.0.0
[7]	Vector	Technical Reference MICROSAR Communication Manager	13.01.00
[8]	Vector	Technical Reference MICROSAR Classic Network Management Interface	13.02.00
[9]	Vector	Technical Reference MICROSAR COM	12.00.00
[10]	Vector	Technical Reference MICROSAR Basic Software Mode Manager	15.01.00
[11]	Vector	Technical Reference MICROSAR EcuM Flex	9.00.00
[12]	Vector	Technical Reference MICROSAR Classic CAN Interface	8.03.01
[13]	Vector	Technical Reference MICROSAR Classic CANTSYN	6.00.00
[14]	Vector	Technical Reference MICROSAR Classic Ethernet Interface	14.03.01
[15]	Vector	Technical Reference MICROSAR PDU Router	8.03.00
[16]	Vector	Technical Reference MICROSAR Classic Socket Adaptor	17.04.00
[17]	Vector	Technical Reference MICROSAR Classic TCPIP	16.05.00
[18]	Vector	Technical Reference MICROSAR WDGIF	1.04.01
[19]	Vector	Technical Reference MICROSAR WDGM	2.00.01
[20]	Vector	Technical Reference MICROSAR Runtime Measurement	8.00.01
[21]	Vector	Technical Reference MICROSAR XCP	6.00.02
[22]	Vector	Technical Reference MICROSAR I-PDU Multiplexer	5.01.00
[23]	Vector	Technical Reference MICROSAR Efficient COM for Large Data	3.00.00
[24]	Vector	Technical Reference MICROSAR Classic StbM	9.01.00
[25]	Vector	Technical Reference MICROSAR Classic TCPIP	16.05.00
[26]	Vector	Technical Reference MICROSAR Runtime Measurement	8.00.01
[27]	Vector	Technical Reference MICROSAR Classic CAN Driver	2.01.00
[28]	Vector	Technical Reference MICROSAR CAN Transport Layer	6.00.00

© 2023 Vector Informatik GmbH



Caution
This document gives an overview on product-wide behavior or use-cases. To ensure correct operation of the product always follow the instructions provided in the technical references and (additionally for safety projects) the safety manual. The content of these documents always replaces/overrides the content from this user manual.

© 2023 Vector Informatik GmbH

Contents

1 Introduction 12

2 Multi-Core Support in AUTOSAR 13
2.1 Terminology and Basic Concepts 14
2.1.1 ECUC Partitions / Cores 14
2.1.2 Base Software Satellites / Proxies 14
2.1.2.1 Satellites 15
2.1.2.2 Proxies 16
2.1.3 Multi-Instantiation 17
2.1.4 Core Types 18
2.1.4.1 Master Core 18
2.1.4.2 Slave Core 19
2.1.4.3 BSW Core 19
2.1.5 Partitions 19
2.2 Memory 20
2.2.1 Flash Memory 20
2.2.2 RAM Memory 20
2.2.3 Memory Mapping 21
2.2.4 Caching 21
2.3 OS 22

3 Integration and Startup 23
3.1 Startup Code 23
3.2 Potential Endless-Loops 23
3.3 Pre-OS 24
3.3.1 General 24
3.3.2 EcuM Initialization 24
3.4 Init Task Synchronisation 24
3.5 BSW Initialization 27
3.5.1 BswM 27
3.5.2 Can 27
3.5.3 CanIf 27
3.5.4 CanTp 27
3.5.5 CanTSyn 27
3.5.6 Com 28
3.5.7 ComM 28
3.5.8 Dem 28
3.5.9 EcuM 29
3.5.10 E2E / E2E Xf 29

3.5.11 Eth ... 29
3.5.12 EthIf ... 29
3.5.13 FiM ... 29
3.5.14 IpduM ... 30
3.5.15 LdCom ... 30
3.5.16 NmIf ... 30
3.5.17 Os ... 30
3.5.18 PduR ... 30
3.5.19 Rte ... 30
3.5.20 StbM ... 31
3.5.21 SoAd ... 32
3.5.22 TcpIp ... 32
3.5.23 WdgIf ... 32
3.5.24 WdgM ... 32
3.5.25 veRtm ... 32
3.5.26 Xcp ... 32
3.6 Trusted Function Stubs ... 32
3.7 Hardware Specific Atomic Instructions ... 34

4 Shutdown ... 35
4.1 Background: Shutdown Main Core ... 35
4.2 Background: Shutdown Satellite Core ... 36

5 Configuration ... 37
5.1 BswM ... 37
5.1.1 Partition Synchronization ... 37
5.1.2 Auto Configurations ... 37
5.1.3 Set up for BSW-Split / Multi-Instance CAN/ETH ... 37
5.1.3.1 Problem ... 38
5.1.3.2 Required Behavior ... 39
5.1.3.3 Configuration ... 39
5.1.3.3.1 Preperation ... 39
5.1.3.3.2 Create Initial BswM Configuration ... 39
5.1.3.3.3 Adapt BswM Configuration in Basic Editor ... 40
5.1.4 Request Processing ... 42
5.2 Can ... 42
5.3 CanIf ... 42
5.4 CanTp ... 42
5.5 CanTSyn ... 42
5.6 Com ... 42

5.6.1 Com Main Function Mapping 43
5.6.2 Com PDU – Main Function Mapping 44
5.7 ComM 44
5.8 Dem 45
5.8.1 Dem Task Mapping 46
5.8.2 Dem Data Consistency 47
5.8.3 Det 47
5.8.4 Det Data Consistency 47
5.9 EcuM 47
5.9.1 EcuM Task Mapping 48
5.10 IpduM 48
5.11 Eth 48
5.11.1 ISR OS Application Mapping 49
5.12 EthIf 49
5.13 FiM 50
5.13.1 FiM Task Mapping 50
5.13.2 FiM Data Consistency 51
5.14 LdCom 51
5.15 Nm 51
5.16 Os 52
5.17 PduR 52
5.18 StbM 53
5.18.1 StbM Data Consistency 54
5.19 SoAd 55
5.19.1 SoAd Data Consistency 55
5.20 TcpIp 55
5.20.1 TcpIp Data Consistency 55
5.21 WdgIf 55
5.21.1 WdgIf Data Consistency 56
5.22 WdgM 56
5.23 veRtm 56
5.23.1 veRtm Data Consistency 56
5.24 Xcp 56
5.25 Partition Mapping of SWC / Service SWC 57
5.25.1 SWC Mapping 57
5.25.2 Task Mapping 58
5.26 Rte/SchM Runnable Timing 59

6 BSW-Split 61
6.1 Architecture 62
6.2 General Steps 63

6.3 Integration Example: Ethernet ... 65
6.3.1 Os ... 65
6.3.2 Com ... 65
6.3.3 SoAd ... 66
6.3.4 CanIf ... 66
6.3.5 EcuC ... 66
6.3.6 PduR ... 67
6.3.7 BswM ... 67
6.3.8 ComM ... 68
6.3.9 Nm ... 68
6.3.10 StbM ... 69
6.3.11 Det ... 69
6.3.12 Rte ... 69
6.3.13 Other Modules ... 69
6.3.14 Initialization ... 72
6.3.15 PduR Appl Spinlocks ... 72

7 BSW-Split: Addon Multi-Instance ETH/CAN ... 74
7.1 Architecture: Multi-Instance ETH ... 74
7.2 Architecture: Multi-Instance CAN ... 76
7.3 Configuration ... 77
7.3.1 ComM ... 77
7.3.2 Eth ... 77
7.3.3 EthIf ... 78
7.3.4 TcpIp ... 78
7.3.5 SoAd ... 79
7.3.6 PduR ... 80
7.3.7 NmIf ... 81
7.3.8 BswM ... 81
7.3.9 Can ... 81
7.3.10 CanIf ... 81
7.3.11 CanTp ... 81
7.3.12 OS ... 81
7.3.13 Runtime System ... 82
7.4 Wrapper Components ... 82
7.4.1 Affected APIs ... 84
7.5 Startup-Synchronization ... 85

8 Application SWC Design Recommendations ... 87
8.1 Code Design ... 87
8.2 Inter-Core Data Flow ... 87

8.3 Software Distribution/Mapping 88
8.4 Configuration Aspects 89

9 Glossary and Abbreviations 90
9.1 Glossary 90
9.2 Abbreviations 90

10 Contact 92

Illustrations

Figure 2-1 AUTOSAR Architecture 4.3.0 13
Figure 2-2 BSW Satellites and Proxies 15
Figure 2-3 Overview: Multi-Instantiation 18
Figure 2-4 Partitions 19
Figure 2-5 Scheduling Order (AUTOSAR OS) 22
Figure 3-1 Example Dem Pre-Init 28
Figure 3-2 Dem_Satellite_Init 29
Figure 3-3 Example Trusted Function Call Stub 34
Figure 4-1 Shutdown Main Core 35
Figure 4-2 Shutdown Slave Core 36
Figure 5-1 Multi-Instance Com-Stack 38
Figure 5-2 Example Architecture Configuration 39
Figure 5-3 BswM Configuration – Bus-Partition 40
Figure 5-4 BswM Configuration – Com-Partition 40
Figure 5-5 BswM Configuration – Bus Partition: Basic Editor 41
Figure 5-6 BswM Configuration – Com Partition: Basic Editor 41
Figure 5-7 BswM Configuration – Correct References 42
Figure 5-8 Com General Architecture 43
Figure 5-9 Com Main Function Partition Mapping 44
Figure 5-10 Com PDU Main Function Mapping 44
Figure 5-11 ComM Channel Mapping 45
Figure 5-12 ComM Main Functions (Example) 45
Figure 5-13 Auto Solving Action DemOsApplicationRef 46
Figure 5-14 Dem Task Mapping 47
Figure 5-15 EcuM Partition References for Slave Core Handling 48
Figure 5-16 Eth – VLAN Queue Configuration 49
Figure 5-17 EthIf – Partition Mapping 50
Figure 5-18 FiM Partition Assignment 51
Figure 5-19 StbM Partition Mapping 54
Figure 5-20 SWC Mapping 57
Figure 5-21 EcuC Partition Mapping 58
Figure 5-22 Task Mapping 58
Figure 5-23 Task Mapping Assistant 59
Figure 5-24 Rte/SchM Runnable Timing – Standard Behavior 59
Figure 5-25 Rte/SchM Runnable Timing – Using Rte/SchM_StartTiming 60
Figure 5-26 Rte/SchM Runnable Timing – Auto Configuration: Module Initialization 60
Figure 6-1 BSW-Split Overview 61
Figure 6-2 BSW-Split Vertical Cut Architecture 62
Figure 6-3 Integration Example: Ethernet: Com Ipdu Partition Assignment 66
Figure 6-4 Integration Example: Ethernet: SoAd Instance Partition Assignment 66
Figure 6-5 Integration Example: Ethernet: CanIf Pdu Partition Assignment 66
Figure 6-6 Integration Example: Ethernet: PduR Queue Creation 67
Figure 6-7 Integration Example: Ethernet: PduR Routing Configuration 67
Figure 6-8 Integration Example: Ethernet: BswM Partition Configuration Creation 68
Figure 6-9 Integration Example: Ethernet: ComM Channel Assignment 68
Figure 6-10 Integration Example: Nm Master Partition Assignment 69
Figure 6-11 Integration Example: Det EA Configuration 69
Figure 6-12 BSW-Split – Example RPC Wrapper 71
Figure 7-1 Add-On Multi-Instance ETH/CAN 74
Figure 7-2 BSW-Split Multi-Instance Architecture ETH 75
Figure 7-3 BSW-Split Multi-Instance Architecture CAN 76
Figure 7-4 Multi-Instance ETH/CAN: VLAN to Queue Assignment 78

Figure 7-5 Multi-Instance ETH/CAN: EtIf Controller Mapping 78
Figure 7-6 Multi-Instance ETH/CAN: TcpIp Multi Partition Configuration 79
Figure 7-7 Multi-Instance ETH/CAN: SoAd Instances Configuration 79
Figure 7-8 Multi-Instance ETH/CAN: PDU Partition Mapping 79
Figure 7-9 Multi-Instance ETH/CAN: Individual PDUs for Routing Path 80
Figure 7-10 Multi-Instance ETH/CAN: Shared PDUs for Routing Path with Dedicated Partition Ref 80
Figure 7-11 Multi-Instance ETH/CAN: CanTp SDU Mapping 81
Figure 7-12 Multi-Instance ETH/CAN: Main Function Mapping 82
Figure 7-13 Multi-Instance ETH/CAN: Wrapper Architecture 83
Figure 7-14 Multi-Instance ETH/CAN: Target Instance Determination 83
Figure 7-15 Multi-Instance ETH/CAN: Startup Synchronization with OS Barriers 85

Tables

Table 5-1 Eth – ISR – OS App Mapping 49
Table 6-1 BSW-Split – Example PduR Appl_TryToGetSpinlock 73
Table 9-1 Glossary 90
Table 9-2 Abbreviations 91

1 Introduction

This document is a guideline and knowledge base to help during project setup of a MICROSAR multi-core project. It is structured in five major sections.

1. The first section gives an overview of the general goal of the multi-core support as specified by AUTOSAR, the general terminology and the multi-core features of the MICROSAR base software.
2. The second section describes the steps that are required to set up a multi-core project, respectively migrate from a single-core to a multi-core project. This covers system integration, system startup and shutdown.
3. The third section shows the multi-core-specific configuration parameters of the MICROSAR base software.
4. The fourth section explains the two different distribution use cases of the communication stack (BSW-Split and BSW-Split Multi-Instance).
5. In the fifth and last section, a set of implementation and configuration recommendations for the application SWC is given that take the particularities of the AUTOSAR stack into account.

Depending on the previous knowledge, this document can be used differently. If multi-core is a completely new topic, it is recommended to have a look at section one first (chapter 2). This chapter contains the foundation and explains the terminology which is used in this document.

With elementary knowledge in multi-core, sections 2 to 5 (chapters 3 to 8) of this document can be used ad-hoc as additional source of information during the integration process of the ECU.

This document has two major goals

> Provide the user sufficient background knowledge to understand the topic “multi-core”

> Provide module-spanning knowledge on ECU-system and use-case level (how multi-core modules interact, how they fit into the overall picture)

This document shall not replace other sources of information to this topic. Consequently, the technical reference that comes with a BSW module should always be considered as primary source of information for that specific module.

2 Multi-Core Support in AUTOSAR

This document refers to a multi-core architecture of the AUTOSAR basic software (BSW) according to Release 21-11. In this architecture, the application SWC can be distributed arbitrarily over all available cores while most of the BSW modules remain on the BSW core. Furthermore, the communication clusters can be mapped to arbitrary cores (X-shaped communication). All cores that run AUTOSAR software (aka AUTOSAR cores) can provide additional instances and/or satellites of BSW modules.

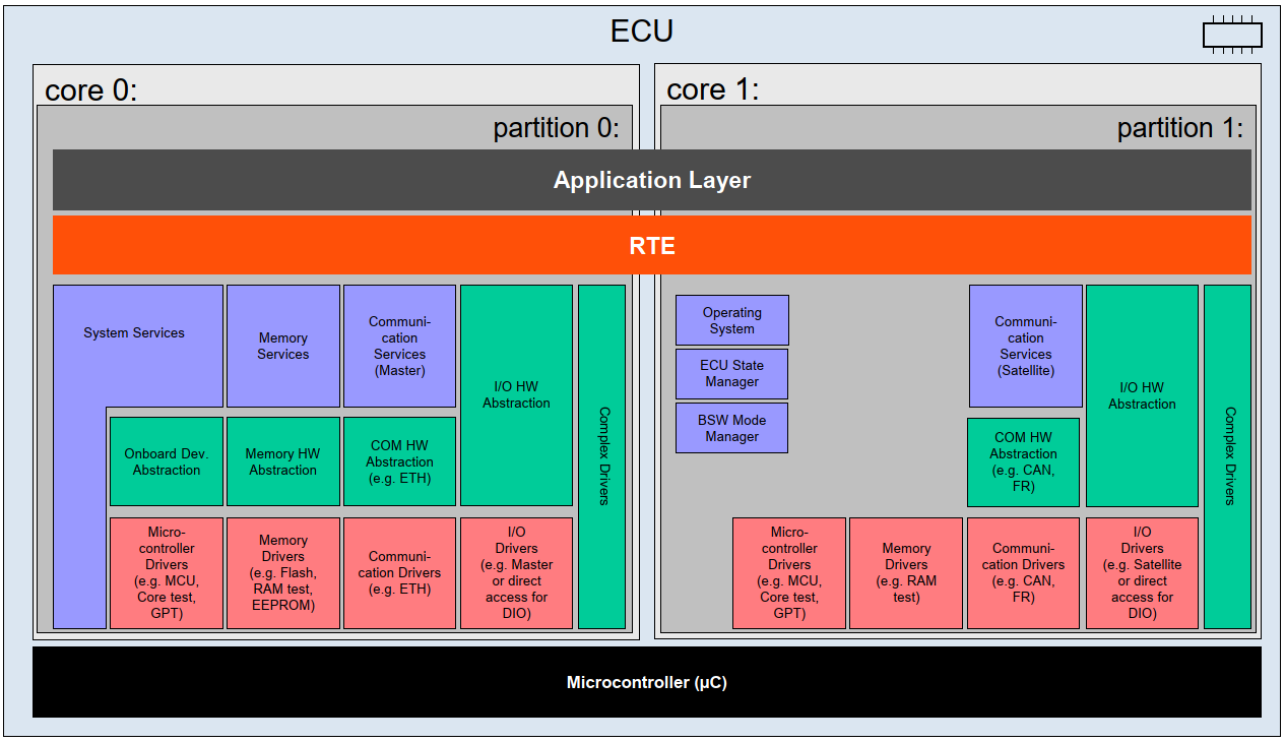


Figure 2-1 AUTOSAR Architecture 4.3.0

The RTE spans over all AUTOSAR cores and provides transparent access to the BSW services from all cores- and partitions. The RTE connects SWCs directly with BSW modules or their satellites. If no satellites are available, the RTE implements mechanisms for crossing the core / partition boundary to the BSW modules.

© 2023 Vector Informatik GmbH



Note
MICROSAR does not support arbitrary core/partition mapping of BSW modules. Instead, MICROSAR offers a couple of core distribution architectures that are described in this document. Any other core/partition crossing between BSW modules must be handled by user code that ensures data consistency and maintains the expected sequential order of API calls.

2.1 Terminology and Basic Concepts

2.1.1 ECUC Partitions / Cores

AUTOSAR uses the terms “Core” and “ECUC Partition” for software deployment and software partitioning. The two terms express different use cases:

> Multi-core: The main goal of using multiple cores is to provide more CPU resources in total (→ Load distribution/balancing)

> Multi-partitioning: The target of having multiple partitions is to achieve safety-separation between different parts of the software (→ segregation in time- and space)

It is not uncommon that both use cases are used in combination. This is natively supported by the MICROSAR BSW and only requires corresponding licenses but not different code.

Due to the fact, that every ECUC Partition requires a core to be executed on, and an ECUC Partition cannot span multiple cores, multi-core is automatically also multi-partitioning. In MICROSAR, the term multi-partitioning is used per default to express, that a component can realize multi-core **and** multi-partition use cases.

2.1.2 Base Software Satellites / Proxies

AUTOSAR describes the architecture pattern of BSW satellites- and proxies as an alternative to remote procedure calls (RPC) in the RTE to server runnables.

Without a satellite, the RTE automatically generates a RPC mechanism for each synchronous C/S port interface between partitions on different cores (Note: This also includes C/S interfaces between two SWC). The following picture shows the basic function principle of both approaches.

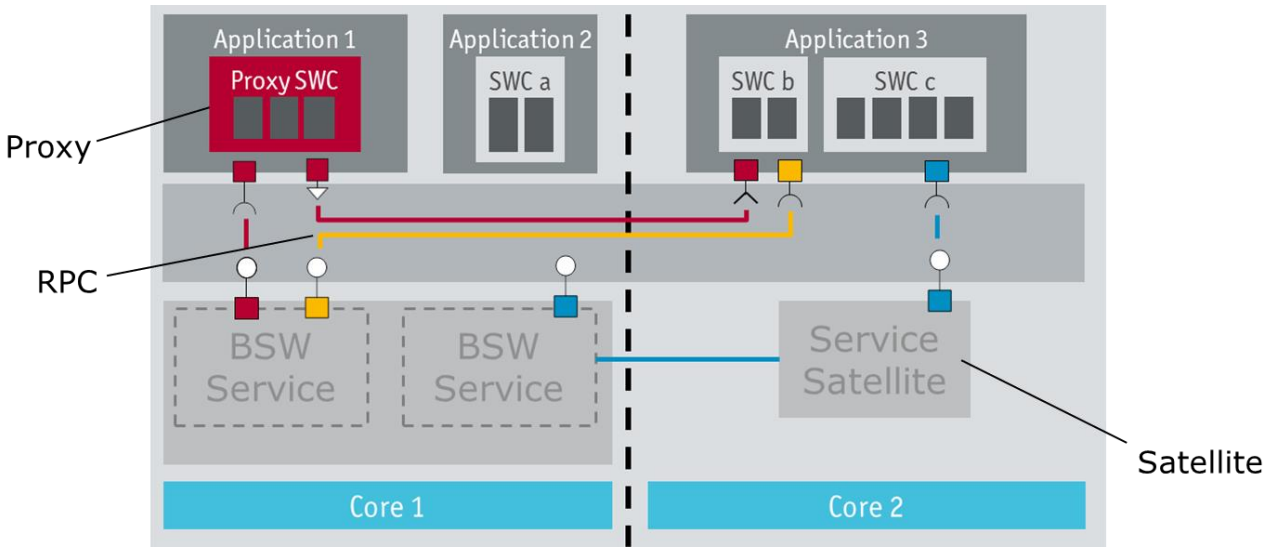


Figure 2-2 BSW Satellites and Proxies

The RPC mechanism is flexible and works in every use case but generates a considerable amount of runtime overhead. The latter was the motivation to introduce satellites and proxies. Both provide execution time efficient access to BSW services on all cores / partitions.

Satellites and proxies relax the architecture recommendation to map all SWC with BSW service needs to the same core as the corresponding BSW service components. Even though Satellites and Proxies add overhead execution time by their internal inter core/partition communication, it is way less to the overhead caused by an RPC.

2.1.2.1 Satellites

A BSW satellite is part of a BSW service module. The idea is that the satellite offers services partition-local, which means that the server runnable is executed in the same core/partition context as the client. The functional scope of satellites ranges from full copies of the service module to very basic functionality that merely transfers the data from and to the master. Common for all satellites is, that they exchange data and/or control flow with the (master) BSW service module. Given the fact that the BSW service module can implement a tailored communication path that optimally suits its communication needs, communication is more efficient compared to the universal path via the RTE (i.e. the RPC mechanism).

Major Benefits of a Master/Satellite architecture are:

> The RTE does not implement RPC for services used by SWCs on different cores (improved efficiency)

> The satellites take over a part of the execution time (→ CPU utilization) of the BSW service since a part of the service's algorithm is executed within the server runnable on the caller's core

MICROSAR realizes a Master/Satellite architecture with the following service components (Service Layer):

> Watchdog Manager (WdgM)

- Diagnostic Event Manager (Dem)
- Function Inhibition Manager (FiM)
- Base Software Mode Manager (BswM)
- Operating System (Os)
- Synchronized Timebase Manager (StbM)
- Universal Measurement and Calibration (XCP)
- Communication (Com)
- Large Data Com (LdCom)

Other BSW modules that implement a Master/Satellite architecture are:

- IPDU-Multiplexer (IpduM)
- ECU State Manager (EcuM)
- Communication Manager (ComM)
- Network Management Interface (NmIf)
- Runtime Measurement (Rtm)



Note
The second group of modules (IpduM, EcuM, etc.) implement multi-core functionality. However, they only offer Service Interfaces to the Application SW-C on one partition (if they offer Service Interfaces at all).

2.1.2.2 Proxies

A proxy translates the services of a BSW service module to a different interface type. The idea is to substitute the execution time expensive cross-core client/server operations with more lightweight sender/receiver Port Interfaces. Hence, whenever a SWC uses a service, it merely reads or writes data in shared RAM (RTE) instead of triggering an RPC.

For this, a proxy provides two sets of APIs:

- The standard C/S interface towards the actual BSW service
- A S/R interface towards the application SWC

The proxy is mapped to the same core as the respective BSW component and uses the C/S interfaces towards the BSW.

Application SWCs communicate via the S/R interfaces with the proxy. core/partition boundaries between SWC and proxy are handled by the RTE.

MICROSAR offers two proxies that do not have to be implemented manually as part of its feature-set. Beside those two, the proxy pattern can be applied to arbitrary services or between two SWC. The proxies must in those cases however be implemented manually.

Prerequisite for using proxies is, that the SWC uses S/R type interfaces. SWC must therefore be designed and implemented accordingly.

The available, automatically generated proxies of MICROSAR are:

- Communication (Com)
- Non-Volatile Memory Manager (NvM)

The strategy that describes when the proxy calls the actual C/S operation of the service depends on the service itself. E.g.:

Com-Proxy

- Transmission: Immediately before the execution of the `Com_MainFunctionTx` (scheduling must be configured correspondingly during integration)
- Reception: In the context of the `Com_MainFunctionRx`

NvM-Proxy. Configurable:

- Cyclic **or**
- After change **or**
- At shutdown
- …

2.1.3 Multi-Instantiation

In MICROSAR, the term “Multi-Instantiation” is used to express different things:

1. By MCAL drivers to tell, that there are multiple instances of a kind of driver for different physical hardware (e.g. drivers for an internal and an external watchdog).
2. By the communication stack to express, that the modules are being executed in multiple partitions for CPU load balancing reasons. A partition that contains an instance executes the module’s main functions, interrupt service routines (if there are any) and APIs.

In this document, “Multi-Instantiation” always means the latter of the two definitions. MICROSAR supports individual instances for selected microcontroller platforms for:

- CAN channels
- VLANs

This means, a CAN channel and a VLAN can be mapped to an ECUC partition. The processing for Rx and Tx will get done in the context of that partition. This opens another degree of freedom for CPU-load balancing of the BSW on top of BSW-Split.

Multi-instantiation does not mean to have multiple copies of a BSW module in the RAM and ROM of the microcontroller. Instead, a multi-instance module consists of partition specific and partition unspecific objects:

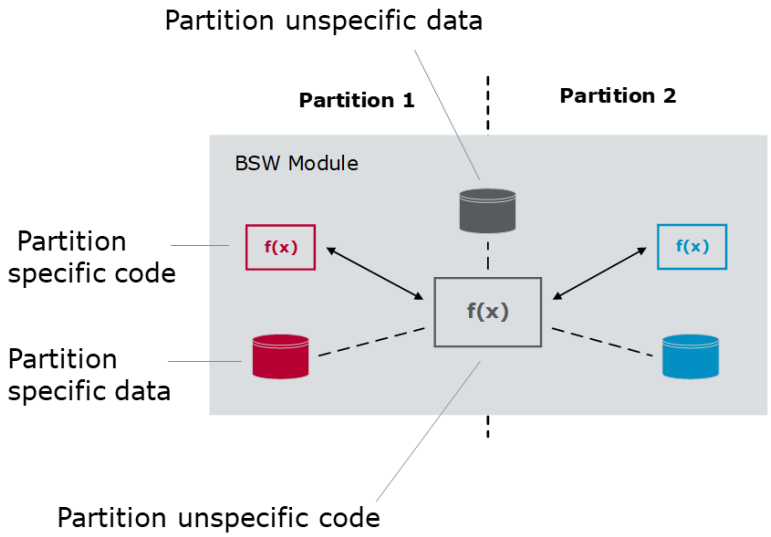


Figure 2-3 Overview: Multi-Instantiation

- Partition-specific
  - **Code** that is exclusively used by one instance. For example, a main function.
  - **Data** that logically (part of an array/struct) or physically (individually locatable memory object) belongs to an instance.
- Partition-unspecific
  - **Code** that is being executed by multiple or all instances
  - **Data** that is read or written by multiple or all instances

In MICROSAR it is module-specific how the exact distribution into specific and unspecific objects looks like.

Note
Please refer to the module's technical reference for instructions on the deployment/location of the module's objects.

Note
Multi-Instantiation requires an addon-license to BSW-Split.

2.1.4 Core Types

2.1.4.1 Master Core

The master core of the microcontroller is started automatically after power-on. It executes its startup code (RAM, MCU initialization etc.) and finally calls the function `EcuM_Init`.

There are controllers on which more than one core are started automatically after power-on. On those, AUTOSAR requires the user to provide integration code that enforces the specified startup behavior (blocking of slave cores). Please refer to chapter 3.1 for more details.

2.1.4.2 Slave Core

A slave core is not started up automatically after power-on. Instead, it gets started by another core during the power-on sequence. Like the master core, it executes its startup code and finally calls the function `EcuM_Init`.

2.1.4.3 BSW Core

BSW core was a term in AUTOSAR and MICROSAR before the specification of the X-shaped communication architecture. Back then the BSW (master) modules have been executed on the BSW core .

However, this definition does not really hold anymore since BSW modules can be executed exclusively on different cores and there are many more modules that implement a master-satellite architecture which effectively are mapped to multiple cores.

Also, the rule that the master-part of all modules are mapped to the same core is not necessarily true. The Dem master for example requires to be mapped to the partition with the highest ASIL. It does not matter if this partition is on the BSW core or not.

For all those reasons, it does not really make sense to use the term BSW core as a strict definition. Instead, the term shall state which core has the most BSW mapped to or which has been dedicated by the integrator as such.

2.1.5 Partitions

In AUTOSAR, partitions are container which contain a set of application SWC and/or base software modules (e.g., satellites). Partitions have SIL ranging from QM up to ASIL D and have dedicated memory protection sets.

Several partitions can share a core, thus allowing mixed ASIL software to be executed on one core.

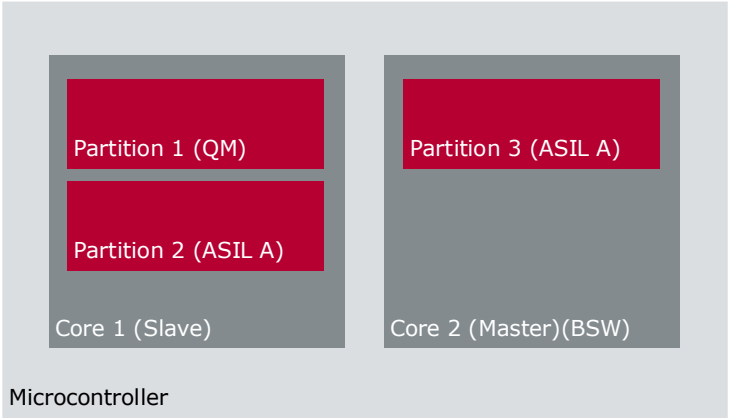


Figure 2-4 Partitions

In AUTOSAR, partitions are realized by **OS Applications**. Therefore, these two terms are typically used synonymously.

2.2 Memory

Modern microcontroller architectures feature several different memories. The most important types are flash memory for persistent data storage and RAM memory for transient program and data storage. Memory mapping in AUTOSAR mainly focuses on practical (e.g. performance and size) and safety relevant criterions.

2.2.1 Flash Memory

Flash memory contains BSW and application SWC code together with their constants and text symbols.

Modern microcontrollers provide several flash memories, that enable advanced use cases for over-the-air update (A-B-switch) but also for segregation and/or performance optimization.

For performance reasons, it is highly recommended to map code and data used by one core to flash memories that are exclusive to that core or that have an exclusive physical access for that core.



Note
Please refer to the BSW Modules' Technical References to get more information on which code/data objects can be mapped to core specific memories.

2.2.2 RAM Memory

RAM memory is primarily used to store variable data and the stack memories although flash bootloaders also use RAM to store executable code.

Modern microcontrollers have several physical RAM units. The number and size of these units greatly depend on the microcontroller respectively its derivative. Nevertheless, RAM can in principle be categorized into:

- **Core local RAM:** Core local RAM is, as the name implies, tightly coupled to a processor core. Usually, the core can access this memory without using the microcontroller's interconnect and thus only suffers from least latency caused by memory-induced wait states. Despite this tight coupling, those RAMs are usually accessible by foreign cores as well. This may however be restricted by the memory protection unit (MPU).
- **Global RAM:** Global in this context means that such RAM is not assigned to a specific microcontroller core (or peripheral). Instead, it is connected to the microcontroller's interconnect and thus access is shared between all bus masters (core, DMA). Typically, this RAM has a higher access latency compared to core local RAM. Global RAM is often not cached, which qualifies it as shared memory for inter-core data exchange.

  Microcontrollers may have several global RAM units. Each of them is connected individually to the interconnect and, depending on the microcontroller architecture, might not be accessible with the same latency from every core.



Note
Please be aware that a memory might be considered shared although it is located to a core local physical RAM.

The core local physical memory of a microcontroller can in most cases and depending on the MPU settings contain shared objects (variables).

The same is true for global physical memory that can contain core-specific non-shared objects.

2.2.3 Memory Mapping

AUTOSAR allows a very fine-grained mapping of memory symbols to linker sections and hence addresses in the µCs memory. Memory symbols are basically either code, text, or data. Data symbols are divided into subgroups in dependence of their type (constants or variables) and size. There are subgroups for each BSW module and for each SWC. Therefore, it is easy to map all memory symbols of a BSW module or an application SWC to specific memory sections.

Rationales for such a fine-grained mapping are:

- **Safety**: Data is mapped to memory sections on which only trustworthy partitions have write access by the MPU.
- **Performance**: Mapping of symbols which are only or mostly used by a single core to a dedicated memory unit (RAM, flash) minimizes interference with other cores during memory access and thus reduces latency.



Note
Despite the still improving technology and architecture refinements, memory mapping is more important than ever to achieve the best performance with a microcontroller. A well-thought-out memory mapping that respects particularities of the controller's memory units is a must since wait states caused by accessing a memory via the interconnect or conflicts when multiple cores try to access the same memory at the same time quickly sum up and slow down the whole system.

2.2.4 Caching

Instruction- and data caching is a feature that almost all modern microcontrollers have. On some of them, it is however not active after power-on/reset.

For MICROSAR it is highly recommended to activate instruction caching since it greatly speeds up flash accesses.

Without instruction caching, execution gets slowed down by wait states that are added when the flash memory must serve multiple cores simultaneously and/or the flash has a lower clock frequency than the CPU.

Data caching offers further performance potential. However, it easily leads to non-working or incorrectly working software because changes to a data element are cached and hence not propagated to all cores (in time).

Caution
It is recommended to deactivate data caching in conjunction with the MICROSAR stack.

2.3 OS

The AUTOSAR OS is an Asymmetric Multi-Processing (AMP) system with the different OS entities statically bound to cores.

However, only OS Applications are mapped explicitly to cores during configuration. Other entities like Tasks, ISRs, Alarms and Schedule tables are mapped indirectly via references from OS Applications.

Task scheduling is done core-locally by the different OS instances. It is nevertheless possible to set events, start tasks etc. on different cores and thus influence the other core's schedule. The priority-based scheduling order is core-local and regards only tasks that are mapped to the same core. There can be, for example, an idle task running on one core, while several higher prior tasks are waiting on another core.

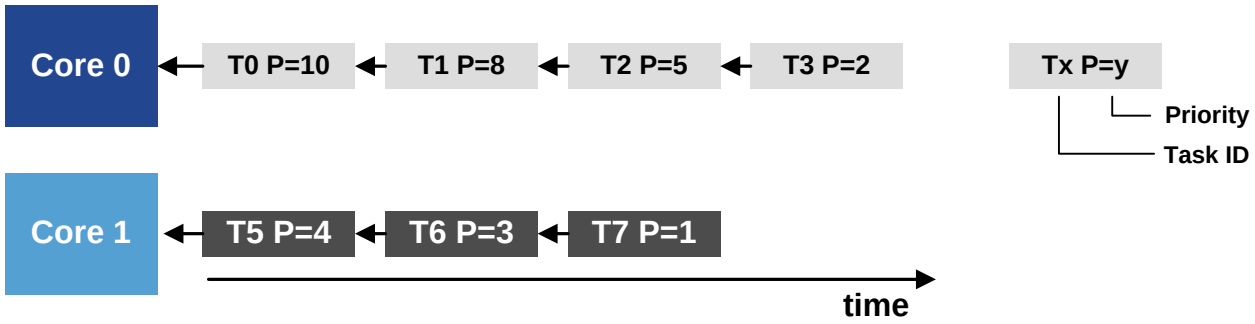


Figure 2-5 Scheduling Order (AUTOSAR OS)

For this reason, the definition of a specific program sequence that spans over multiple cores is not possible by setting task priorities only A task-sequence can be defined by using schedule tables. Still, such sequences are not enforced by the OS. The OS merely activates tasks- or sets events at predefined points in time. If the tasks get scheduled or not depend on the priorities of the currently running- and waiting tasks.

3 Integration and Startup

3.1 Startup Code

The startup code is executed before a core calls the C `main()` function. Tasks of the startup code are:

- µC configuration (PLL, memory wait states, FPU …)
- Register initialization
- Base address initialization (interrupt and trap vector tables)
- Stack pointer initialization
- Watchdog configuration
- Memory initialization (initialized variables and `zero` init)
- …

On multi-core controllers, all cores need to execute a startup code. For most cases, it is advised to implement a dedicated code for each core. Otherwise, unexpected behavior is likely to occur since all cores use the same stack pointer and tasks like memory initialization or watchdog configuration are done by two or more cores consecutively or simultaneously.

Nevertheless, it makes sense to write the code so that it behaves core-specific, and it can be reused. The core-context can, for instance, be determined by passing it as function parameter. Examples for such code can be found in the BRS.

Additionally, the startup code may need to fulfill the AUTOSAR requirement that only one core shall start up automatically after power-on. All other cores are subsequentially started by the BSW (respectively OS) during its startup. If the microcontroller cannot support this behavior natively, it must be emulated by the startup code. The startup code must prevent all cores except the master core from leaving the startup code until the OS was started on the master core by calling `StartOS()`

3.2 Potential Endless-Loops

In the MICROSAR stack, the system can get stuck in endless loops in case of hardware faults and/or incorrect behavior of the software:

1. **StartCore() API**: On some hardware platforms, cores get started by writing to a peripheral register. The platform specific StartCore() API returns as soon as the core was started successfully. This is usually ensured by a loop which checks the status of the core to be started via a hardware status-register. If the hardware does not behave as specified, the StartCore() API gets stuck in this loop.
2. **OS Startup/Shutdown**: The OS uses Synchronization Barriers in the startup- and shutdown process of AUTOSAR cores. The system gets stuck if not all cores arrive at the synchronization points.
3. **BswM Startup:** The BswM uses Synchronization Barriers in its startup process. The system gets stuck if not all cores arrive at the synchronization points.

The potential occurrence of such endless-loops must be considered in the safety analysis of the system.

3.3 Pre-OS

3.3.1 General

The first function which is being executed after the startup code is the main function. This can either be a dedicated function for each core or one common function which branches in dependence to the core context. This context can be determined with the OS API `GetCoreID()`.

An example of such an implementation can be found in the technical reference of the OS [4]. The same document also describes which OS APIs need to be called by which core during startup.

The file `EcuM_Callout_Stubs.c` contains a working example implementation of the code required to start the OS. The example also shows how the slave cores are started by the master core of the microcontroller.



Example

```
FUNC(void, ECUM_CODE) EcuM_StartOS(AppModeType appMode)
{

#if ( ECUM_NUMBER_OF_CORES > 1 )
  uint8 coreId;
  StatusType status;
  if(GetCoreID()==ECUM_CORE_ID_STARTUP)
  {
    for(coreId=0; coreId < ECUM_NUMBER_OF_CORES; coreId++)
    {
      if(coreId!=ECUM_CORE_ID_STARTUP)
      {
        StartCore(coreId, &status);
      }
    }
  }
#endif

  /* Start OS must be called for each core */
  StartOS(appMode);

  return;
}
```

3.3.2 EcuM Initialization

In multi-partition configurations with different ASIL, the function `EcuM_StartupTwo` must be called on a QM partition.

3.4 Init Task Synchronisation

Multi-core systems require more than one `Init_Task`. Depending on the system configuration one of the two options apply:

> If the system contains multi-partition BSW modules (e.g. Dem): One `Init_Task` per OS Application is required

> Otherwise: One `Init_Task` per core is sufficient

The `Init_Task` must be configured with a sufficiently high priority and shall start up automatically, so that they get executed right after OS start, almost simultaneously on all cores.

Nevertheless, certain APIs like `Rte_Start()` must be called in the correct sequence on the different cores respectively OS Applications. Therefore, wait-points are required within the init tasks. These wait points can be realized with the barrier-feature of the Gen7 OS or with global, non-cached variables.



Example 1: Synchronization with Variables

```
volatile uint32 gBarrierA = 0;
volatile uint32 gBarrierB = 0;

TASK(Init_Core_0)
{
    /* Initialization I (Core 0) */

    while( !gBarrierB )
    {
        ; /* Wait for Init_Core_1 */
    }
    gBarrierA = 1;

    /* Initialization II (Core 0) */
}

TASK(Init_Core_1)
{
    /* Initialization I (Core 1) */

    gBarrierB = 1;

    while( !gBarrierA )
    {
        ; /* Wait for Init_Core_1 */
    }

    /* Initialization II  (Core 1) */

}
```

Example 2: Synchronization with OS Barriers

```
TASK(Init_Task_OsCore0)
{

  /* Initialization I (Core 0) */

  Os_BarrierSynchronize(MYSTARTUPBARRIER);

  /* Initialization II (Core 0) */

}

TASK(Init_Task_OsCore1)
{
  /* Initialization I (Core 1) */

  Os_BarrierSynchronize(MYSTARTUPBARRIER);

  /* Initialization II (Core 1) */

}
```



Caution
Other synchronization approaches may also be possible. They must, however, ensure that no application or BSW task is scheduled before the init tasks have completed.

Furthermore, interrupts must be enabled on a per-core basis. This is done by using the OS APIs

- `Os_InitialEnableInterruptSources()`: To generally enable CAT 2 interrupts on a core
- `Os_EnableInterruptSource()`: To enable a specific interrupt (if the interrupt source is not configured to get enabled automatically)

Whether- and how exactly interrupts must be enabled manually is described in the technical reference of the OS.

3.5 BSW Initialization

This chapter describes the initialization particularities of the multi-core/multi-partition BSW modules.

3.5.1 BswM

BswM initialization is done via calls to `BswM_PreInit` and `BswM_Init`. Both calls are done by the EcuM. The first one by the `DriverInitListZero` and the latter one in the context of `EcuM_StartupTwo`.

Details to the initialization can be found in the technical reference of BswM ([6]).

3.5.2 Can

Can is initialized by calling the function `Can_Init()` on exactly one partition. To avoid any issues that arise when Can is used on a partition before it was initialized, Can_Init() shall be called on the Partition that starts first.

Further information can be found the technical reference of Can ([27]) chapter **Multi Partition**.

3.5.3 CanIf

CanIf is initialized by calling the function `CanIf_Init()` on exactly one partition. To avoid any issues that arise when CanIf is used on a partition before it was initialized, CanIf_Init() shall be called on the partition that starts first.

3.5.4 CanTp

CanTp requires a staged initialization on all partitions that handle a CanTp channel.

Details on which API must be called when and in which partition can be found in the technical reference of CanTp ([28]).

3.5.5 CanTSyn

CanTSyn requires a staged initialization on all partitions that contain a CanTSyn instance.

Details on which API must be called when and in which partition can be found in the technical reference of CanTSyn ([13]) chapter **Initialization**.

3.5.6 Com

Com is 28ynchroniza by calling the function `Com_Init()` on exactly one partition. Details to the initialization can be found in the technical reference of Com ([9]).

3.5.7 ComM

ComM requires a synchronized, multi-step initialization on all partitions as described by the technical reference of ComM ([7]).

3.5.8 Dem

The Dem requires a synchronized, multi-step initialization on all partitions as described by the technical reference of Dem ([1]).

Additionally, this chapter describes some hints and guidelines as well:

Pre OS (Driver Init One):

Since this initialization step is performed before the OS is started, there is no OS Application specific execution context. It is therefore valid to pre-initialize the satellites on a per-core base. This means, the satellites of all OS Applications that are assigned to one core can be pre-initialized by this core.

The assignment of satellite ID and OS Application can be found in the Dem configuration (`Dem_Lcfg.c`) in table: `Dem_Cfg_SatelliteInfo`.

Figure 3-1 shows a simple example with the following preconditions:

> There is exactly one Dem satellite per core

> The Dem satellite ID equals the OS Application ID which in turn equals the core ID: `GetApplicationID()` == `GetCoreID` == Dem satellite ID

```
/*********************************************************************************************************************
 * EcuM_AL_DriverInitOne
 *********************************************************************************************************************/
FUNC(void, ECUM_CODE) EcuM_AL_DriverInitOne(void)
{
  if(GetCoreID() == ECUM_CORE_ID_BSW)
  {
      Dem_MasterPreInit(Dem_Config_Ptr);
  }
  /* In this configuration, there is only one OS Application per core and the core IDs correspond to the satellite IDs */
  Dem_SatellitePreInit( GetCoreID() );
```

Figure 3-1 Example Dem Pre-Init

Partition Specific Init-Task

Synchronization across multiple `Init_Tasks` can be achieved as described in chapter 3.4. For the `Dem_SatelliteInit()` calls it is sufficient to execute them in consecutive scheduled init-tasks without any further call-order.

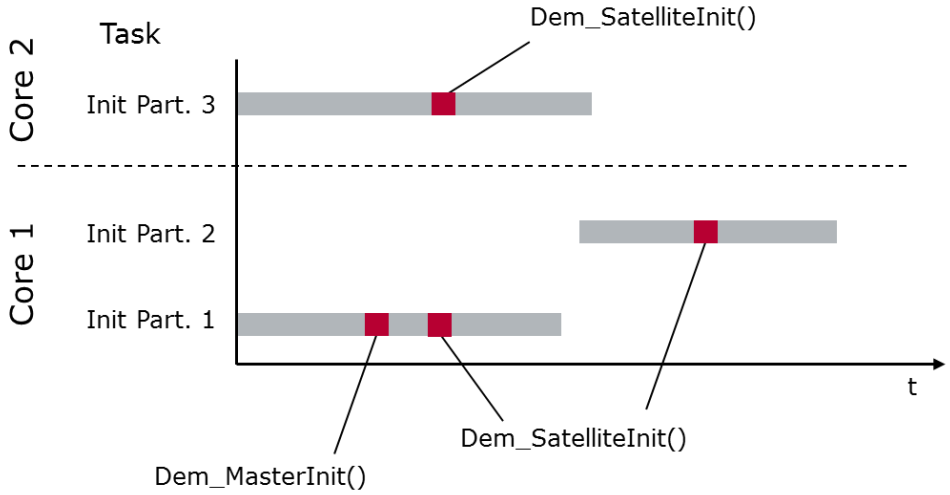


Figure 3-2 Dem_Satellite_Init

Mixed ASIL Configurations

The MPU must be set-up appropriately to allow the necessary memory write access to memory section group “Restricted” for this step (please refer to [3]). This can be achieved with a Pre-Start Task of the MICROSAR OS (please refer to [4]).

3.5.9 EcuM

EcuM is 29ynchroniza by calling the functions `EcuM_Init()` and `EcuM_StartupTwo()` on all cores (in the context of the partitions referenced in `EcuMPartitionRef`). More information on the initialization can be found in the technical reference of EcuM ([11]).

3.5.10 E2E / E2E Xf

In multi-partition configurations with different ASIL, the following function must be called on the highest ASIL partition:

`E2Exf_Init()`

This can be done via the BswM on the responsible partition or by integration code in the context of the init-task.

3.5.11 Eth

Eth is initialized by calling the function `Eth_<HW>_Init()` on exactly one partition. To avoid any issues that arise when Eth is used on a partition before it was initialized, `Eth_<HW>_Init()` shall be called on the partition that starts first.

3.5.12 EthIf

EthIf is initialized by calling the function `EthIf_Init()` on exactly one partition. To avoid any issues that arise when EthIf is used on a partition before it was initialized, `EthIf_Init()` shall be called on the partition that starts first.

3.5.13 FiM

FiM requires a synchronized, multi-step initialization on all partitions as described by the technical reference of FiM ([5]).

3.5.14 IpduM

IpduM is initialized by calling the function `IpduM_Init()` on exactly one partition. To avoid any issues that arise when IpduM is used on a partition before it was initialized, `IpduM_Init()` shall be called on the partition that starts first.

More information on the initialization can be found in the technical reference of IpduM ([22]).

3.5.15 LdCom

LdCom is initialized by calling the function `LdCom_Init()` on exactly one partition. To avoid any issues that arise when LdCom is used on a partition before it was initialized, `LdCom_Init()` shall be called on the partition that starts first.

More information on the initialization can be found in the technical reference of LdCom ([23]).

3.5.16 NmIf

NmIf requires a synchronized, multi-step initialization on all partitions as described by the technical reference of NmIf ([8]).

3.5.17 Os

The initialization of the OS is described in detail in the Technical Reference of the OS ([4]), chapter **Startup Concept**.

3.5.18 PduR

PduR requires a synchronized, multi-step initialization on all partitions as described by the technical reference of PduR ([15]).

3.5.19 Rte

The Rte is initialized on a per-core basis. It expects a startup behavior in which the API `Rte_Start()` is called on all satellite cores before it is executed on the main core.

Furthermore, dependent on the configuration, it might be required to postpone scheduling of SchM and application tasks on satellite cores until `Rte_Start()` on the main core has returned. This can for instance be achieved by setting the satellite-cores' init-tasks to the highest priority and remain within these tasks until the main core has returned from `Rte_Start()`.

The required synchronization can be realized as described in chapter 3.4. It is recommended to call `Rte_Start()` manually also on the main core instead of calling it via the BswM. Rationale is that the required call sequence can be achieved easier and more robust if all calls are done manually.



Example

```
TASK(Init_Task_Main_Core)
{
    EcuM_StartupTwo();

    /* Wait for startup of satellite core (Core 1) */
    Os_BarrierSynchronize(STARTUPBARRIER_PRE_RTE);

    /* Manual start of RTE. Autostart of RTE in EcuM must be disabled */
    Rte_Start();

    /* Optional: Post RTE-start core 31ynchronization */
    Os_BarrierSynchronize(STARTUPBARRIER_POST_RTE);

    (void)TerminateTask();
}

TASK(Init_Task_SatelliteCore_1)
{
    /* Manual start of RTE */
    Rte_Start();

    Os_BarrierSynchronize(STARTUPBARRIER_PRE_RTE);

    /* Optional: Post RTE-start core 31ynchronization */
    Os_BarrierSynchronize(STARTUPBARRIER_POST_RTE);

    (void)TerminateTask();
}
```

The code above shows an example of the synchronization of the Rte start on the main core.



Caution

Please be aware that the scheduling and the alarms are active during the post RTE-start synchronization and tasks with lower priority cannot be executed during this time. In case of extensive waiting times, this can lead to OS errors caused by multiple task activations.

Possible remedies are:

- Reduce waiting time during synchronization (between `Rte_Start()` and termination of the init-task)
- Delay scheduling of application tasks via activation offsets

3.5.20 StbM

StbM requires a synchronized, multi-step initialization on all partitions as described by the technical reference of StbM ([24]).

3.5.21 SoAd

SoAd requires a synchronized, multi-step initialization on all partitions as described by the technical reference of SoAd ([16]).

3.5.22 TcpIp

TcpIp is initialized by calling the function `TcpIp_Init()` on exactly one partition. To avoid any issues that arise when TcpIp is used on a partition before it was initialized, `TcpIp_Init`() shall be called on the partition that starts first.

More information on the initialization can be found in the technical reference of TcpIp ([25]).

3.5.23 WdgIf

WdgIf does not require any initialization.

3.5.24 WdgM

Initialization of the WdgM is on a per-core base. It is only required on cores where the WdgM is mapped to.

For initialization, the API `WdgM_Init()` must be called after the OS has started. E.g. in the core's `init` task.



Caution
In multi-partition configurations with different ASIL, WdgM_Init() must be called on a ASIL partition.

3.5.25 veRtm

veRtm is initialized by calling the function `Rtm_Init()` on exactly one partition. To avoid any issues that arise when Rtm is used on a partition before it was initialized, `Rtm_Init`() shall be called on the partition that starts first.

More information on the initialization can be found in the technical reference of veRtm ([26]).

3.5.26 Xcp

Xcp is initialized by calling the function `Xcp_Init()` on exactly one partition. To avoid any issues that arise when Xcp is used on a partition before it was initialized, `Xcp_Init`() shall be called on the partition that starts first.

More information on the initialization can be found in the technical reference of Xcp ([21]).

3.6 Trusted Function Stubs

In partitioned systems, trusted function calls are required for function calls where the caller belongs to a partition with lower ASIL than the called function.

The Rte for example uses trusted functions for client/server, mode, sender/receiver, and Nv communication when the BSW runs trusted and memory from a nontrusted partition needs to be accessed.

Trusted function calls are also used for calls between BSW modules and CDDs if they are mapped to partitions with different safety integrity levels. If such trusted function calls are required or do not depend on the system partitioning.

Trusted function calls are only supported on the same core. Cross-core calls must be implemented manually by using an IOC callback or a worker proxy mapped to a task.

The MICROSAR OS can generate stubs for the trusted functions. If, however a non-MICROSAR OS is used, the trusted function implementation needs to be done manually.

For each trusted function, the following APIs need to be implemented:

```
<ret> Os_Call_<Name Of Trusted Function>( <arg1> … )
```

```
void TRUSTED_Os_ServiceCallee_<Name Of Trusted Function>(
TrustedFunctionIndexType FunctionIndex,
TrustedFunctionParameterRefType FunctionParams )
```

within the OS_Call-function the OS API `CallTrustedFunction()` must be called.
Arguments of this API are:

- ID of the trusted function (from OS configuration)
- Argument(s) of the trusted function

Within the trusted function skeleton, the actual trusted function must be called with the arguments received in the `FunctionParams`.

Example

Here is an example that has been prepared for you.

```
/**********************************************************************************************************************
 *  TRUSTED_Os_ServiceCallee_MyTrustedFunction
 *********************************************************************************************************************/
OS_LOCAL FUNC(void, OS_CODE) TRUSTED_Os_ServiceCallee_MyTrustedFunction
(
  TrustedFunctionIndexType FunctionIndex,
  TrustedFunctionParameterRefType FunctionParams                    /* PRQA S 3673 */ /* MD_Os_3673_FunctionPointer */
)
{
  P2VAR(Os_DummyTrustedFunctionPackageType, AUTOMATIC, OS_VAR_NOINIT) package;
  package = (P2VAR(Os_MyTrustedFunctionPackageType, AUTOMATIC, OS_VAR_NOINIT)) FunctionParams;

  MyTrustedFunction
  (
    package->Arg1
  );

  OS_IGNORE_UNREF_PARAM(FunctionIndex);                                    /* PRQA S 3112 */ /* MD_Os_3112 */
}


/**********************************************************************************************************************
 *  GLOBAL FUNCTIONS
 *********************************************************************************************************************/

/**********************************************************************************************************************
 *  Os_Call_DummyTrustedFunction
 *********************************************************************************************************************/
FUNC(void, OS_CODE) Os_Call_MyTrustedFunction
(
  uint8 Arg1
)
{
  Os_MyTrustedFunctionPackageType package;
  package.Arg1 = Arg1;

  (void)CallTrustedFunction(Os_ServiceCallee_MyTrustedFunction, (TrustedFunctionParameterRefType)&package);
}
```

Figure 3-3 Example Trusted Function Call Stub

3.7 Hardware Specific Atomic Instructions

The MICROSAR stack uses a set of hardware specific atomic instructions for data access- and manipulation to be more runtime efficient. The instructions that are used are either inspired by the AUTOSAR BMC library or are actual APIs of this library.

During integration, those functions can either be provided by a BMC that is suitable to the microcontroller used for the project or in form of manual implemented code.

The following list shows the modules that require atomic instructions. Details can be found in the respective technical reference:

- Dem (→ Atomic Compare/Exchange)
- FiM (→ Atomic Compare/Exchange)
- PduR (→Spinlocks)
- SoAd (→ Bmc_xxx)

4 Shutdown

On multi-core ECUs, the shutdown of the individual cores needs to be synchronized to ensure well-defined application- and BSW behavior in this phase.

Specifically this means that all clean-up and housekeeping jobs of the application SW like storing intermediate data to flash memory or setting I/O pins appropriately has to be done before `EcuM_GoDown()` is called. The recommended way to ensure this sequence is to configure a set of suitable BswM modes (e.g. `RUN`, `PREP-SHUTDOWN` …) which are propagated to the application SW and which determine if a runnable is getting executed or not.

To shut down the ECU, `EcuM_GoDown()` must be called on all cores. The MICROSAR ECUM does take care of that if the following parameter is set to “TRUE”:

`/MICROSAR/EcuM/EcuMGeneral/EcuMSlaveCoreHandling`

From the application’s point of view, the ECU can switch off at any instance after `EcuM_GoDown()` has been called. It is highly recommended to keep the ECU in a consistent and safe state (I/O, memory) until power off/reset.

4.1 Background: Shutdown Main Core

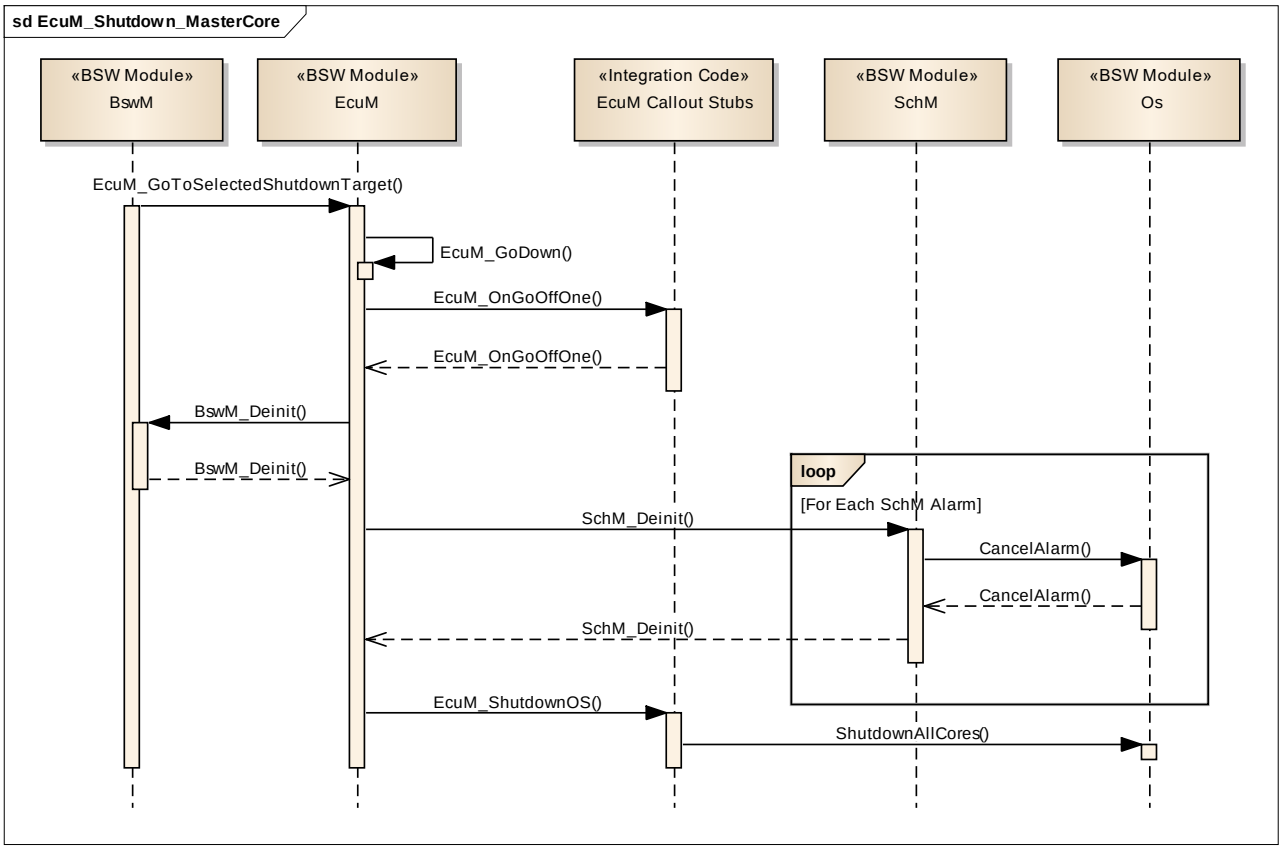


Figure 4-1 Shutdown Main Core

On the main core, the function `EcuM_GoDown()` de-initializes the BswM and stops scheduling of its SchM tasks by cancelling all corresponding alarms. From now on all BSW-

services mapped to this core should be regarded as unavailable since their main functions are not being called any more.

As EcuM's last action, the OS API `ShutdownAllCores()` is called. It synchronizes all cores and ultimately stops the ECUM main- and all satellite cores.

4.2 Background: Shutdown Satellite Core

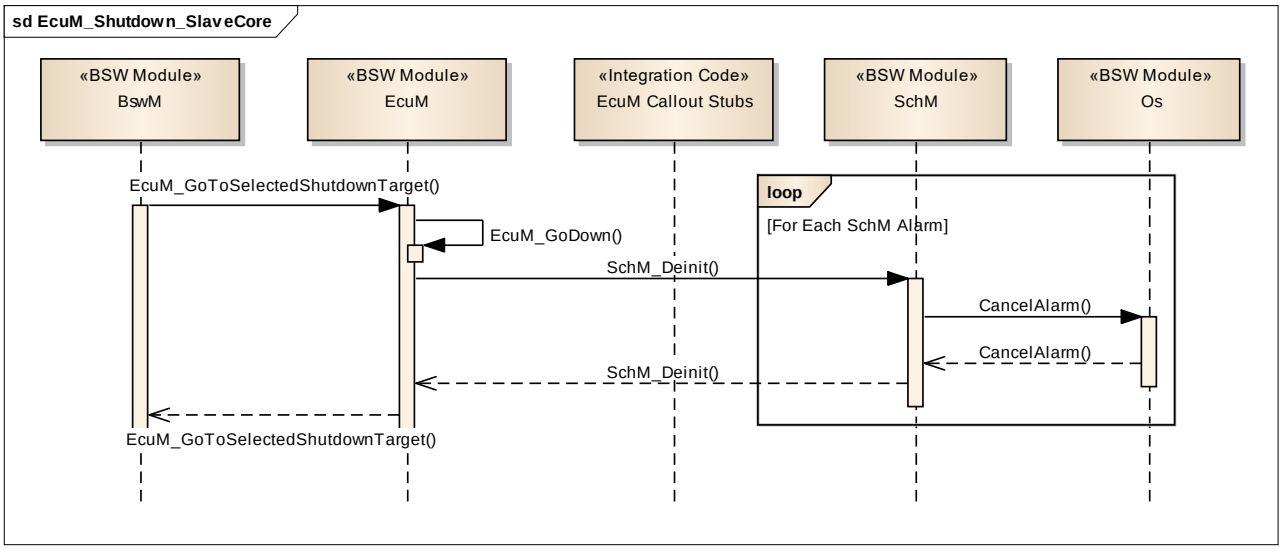


Figure 4-2 Shutdown Slave Core

On the satellite cores, the function `EcuM_GoDown()` merely de-initializes the SchM. Hence, this API only needs to be called if there is a SchM instance on that core.

5 Configuration

This chapter lists a collection of configuration parameters relevant for multi-core systems. It is not a complete list of all parameters but supplements the technical references of the BSW modules and highlights critical and often overlooked parameters.

In any case, the technical reference of a BSW modules is always the primary source of information.

5.1 BswM

In addition to the technical reference of the BswM ([10]), this chapter describes particularities of standard multi-partition and BSW-Split / Multi-Instance CAN/ETH use cases.

5.1.1 Partition Synchronization

The BswM does not synchronize init phases on the different partitions. If a specific temporal order for calling `Init()` functions in different partitions is required, it must be provided and executed as user code after the BswM init-phases.

Synchronisation can be achieved by using

- OS Events (`WaitEvent()`, `SetEvent()`)
- OS Barriers

It depends on the project set-up and requirements which of the above-mentioned mechanisms is better suited.

5.1.2 Auto Configurations

The following limitations must be considered when using the Auto Configurations:

- **ECU State Handling:** Is only supported on one partition. Shall be done on the partition that contains the ComM master
- **Communication Control:** Must be done for a channel in the same partition as it is mapped to.

5.1.3 Set up for BSW-Split / Multi-Instance CAN/ETH

This chapter explains how to set up the BswM configuration in projects that use multiple Instances of:

- Com
- Can
- Eth

The reason for the specific configuration as described in this chapter is the availability respectively scope of Com-, BswM and <bus>_SM APIs.

The following requirements must be adhered:

- The following Com APIs must only be called by the BswM in the **same OS Application** as the ComIpdu Group is mapped to
  - `Com_EnableReceptionDM`

- Com_DisableReceptionDM
- Com_IpduGroupStart
- Com_IpduGroupStop

- The BswM must only be invoked by a <bus>_SM in an OS Application, if it provides the required **BSW Mode Request** Port in this OS Application

5.1.3.1 Problem

Configurations with multiple Com instances **and** multiple <bus>_SM on different OS Applications violate in most cases those requirements and hence must follow this configuration guideline.

The following picture shows this problematic for an example architecture.

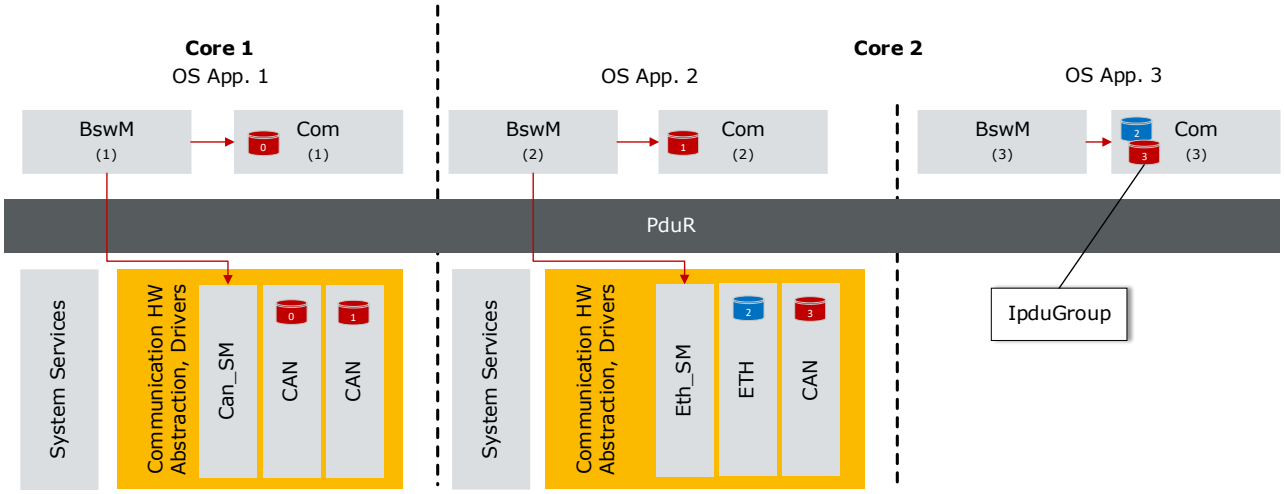


Figure 5-1 Multi-Instance Com-Stack

Note
Please not that this is only an example there are many different deployment variants possible but, in the end, they all have in common, that one of the requirements listed above is violated...

- There are three instances of Can. One of them mapped to a different OS Application (OS App)
  - Can_SM is however not multi-instance capable → It is mapped to one OS App only
- There is one instance of Eth. It is mapped to the same OS App as the Eth_SM.
- There are three Com instances
  - Instance (1) handles IpduGroup 0 of the Can instance on the same OS App. → This automatically binds IpduGroup 0 to OS App. 1
  - Instance (2) handles IpduGroup 1 of the Can instance on a different OS App. Still, IpduGroup 1 is assigned to OS App. 2

> Instance (3) handles the IpduGroups of the Can and Eth instances (2,3) on OS App. 2. Both IpduGroups are bound to OS App. 3.

5.1.3.2 Required Behavior

The required behavior is:

> BswM (1) Provides the BSW Mode Request Port for the Can_SM

> BswM (2) Provides the BSW Mode Request Port for the Eth_SM

> BswM (1) performes the IpduGroup handling with Com (1)

> BswM (2) performes the IpduGroup handling with Com (2)

> BswM (3) performes the IpduGroup handling with Com (3)

5.1.3.3 Configuration

The steps described in this chapter result in a configuration that has the required behavior as described in 5.1.3.2.

The following scenario is used as example:

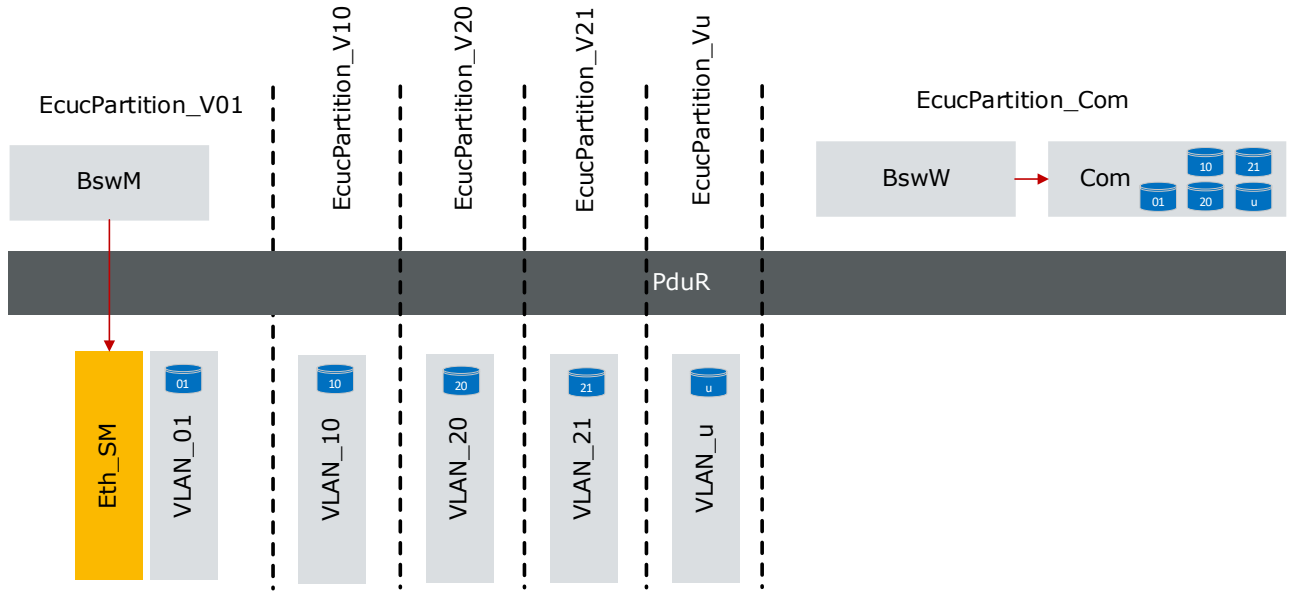


Figure 5-2 Example Architecture Configuration

5.1.3.3.1 Preperation

Set up BSW configuration. Especially ECUC Partitions, Com instances (→ MainFunctions) and BswM instances.

5.1.3.3.2 Create Initial BswM Configuration

Execute the **Configure Communication Control** wizard in **BSW Management** for every Partition:

> That has a <bus>_SM mapped to (Select all those channels that are handled by the corresponding <bus>_SM)

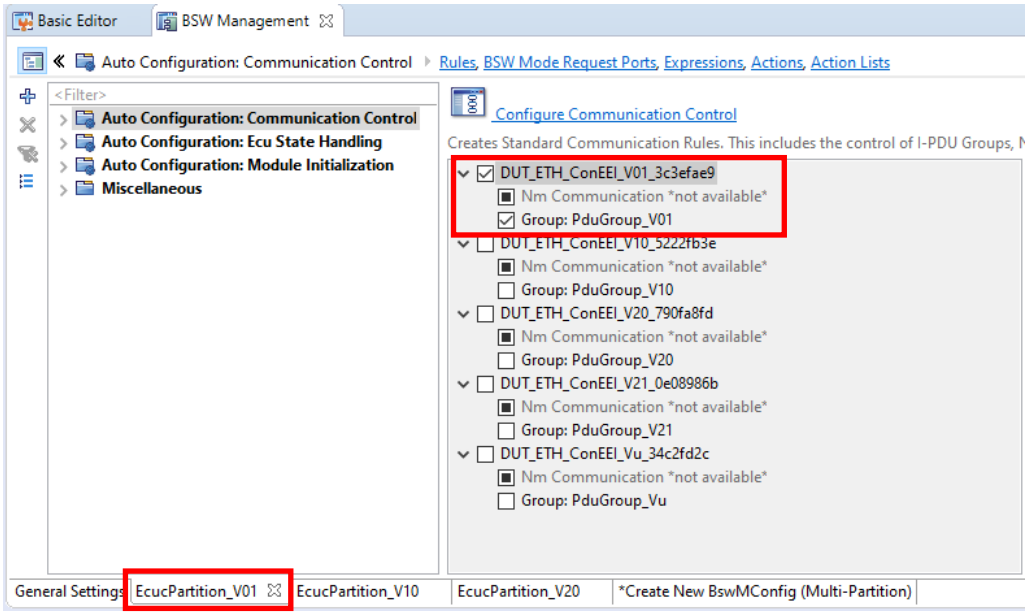


Figure 5-3 BswM Configuration – Bus-Partition

> That has a Com instance which handles one or more IpduGroups (Select all channels/IpduGroups that the Com in this partition handles)

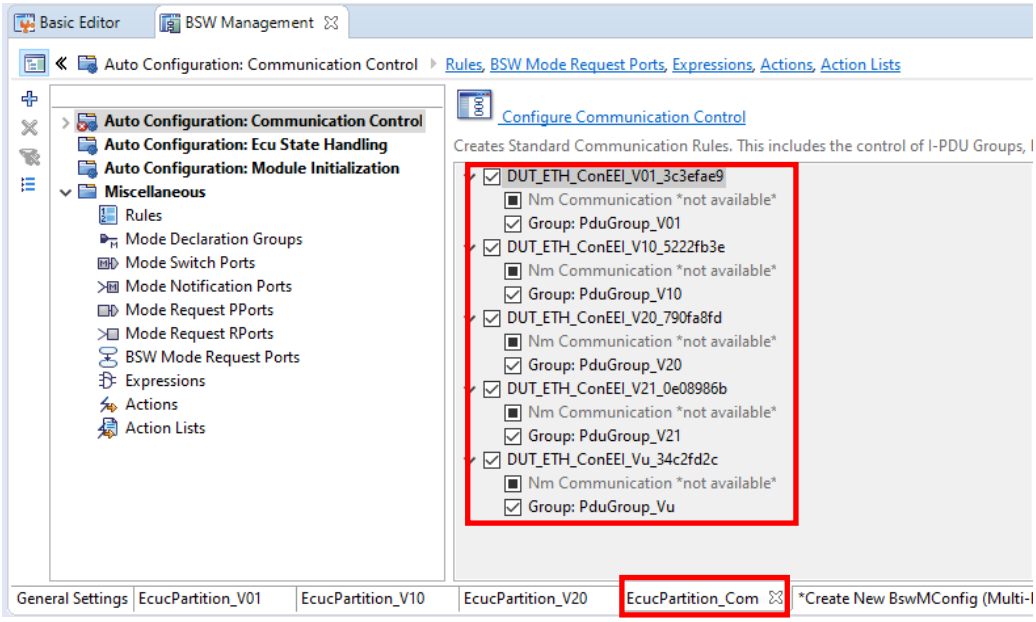


Figure 5-4 BswM Configuration – Com-Partition

Caution
Make sure, that chapter 5.1.3.3.1 is completed since the wizards will overwrite the manual changes done in the later chapters.

5.1.3.3.3 Adapt BswM Configuration in Basic Editor

In the configuration of the **partition** that contains the **<bus>_SM**:

> Delete all configuration items that refer to the handled channel (Dut_Eth_com…) in
  > BswMLogicalExpressions
  > BswMModeConditions
  > BswMRules

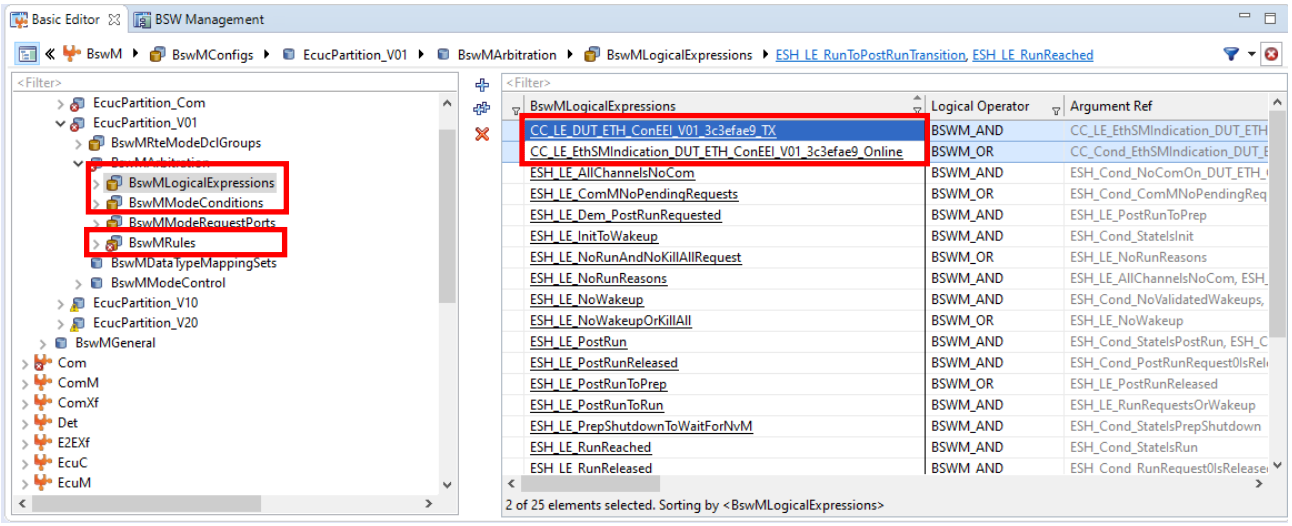


Figure 5-5 BswM Configuration – Bus Partition: Basic Editor

Note
The relevant items start with the prefix “CC_”.

In the configuration of the **partition** that contains the **Com**: Delete the BSW Mode Request Ports of the channels:

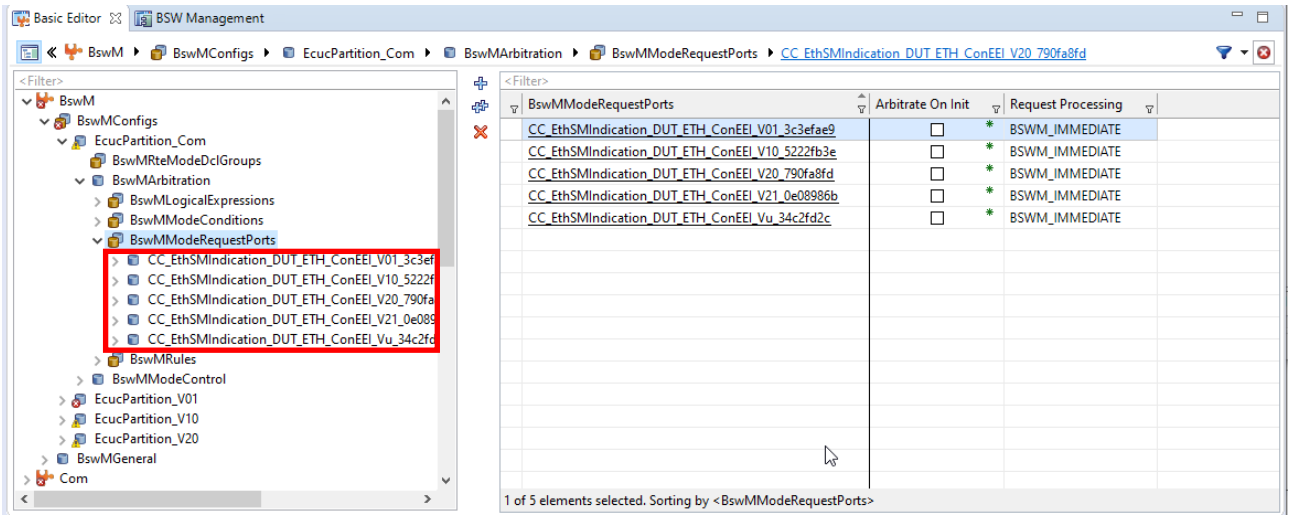


Figure 5-6 BswM Configuration – Com Partition: Basic Editor

> **Fix** the now **broken references** in `BswMModeConditions` by selecting the corresponding Mode Request Port from the Partition that contains the <bus>_SM.

> This can be easily done by copying the (currently) broken `BswMConditionMode` name (string after the last backslash) and use it to search the correct reference target:

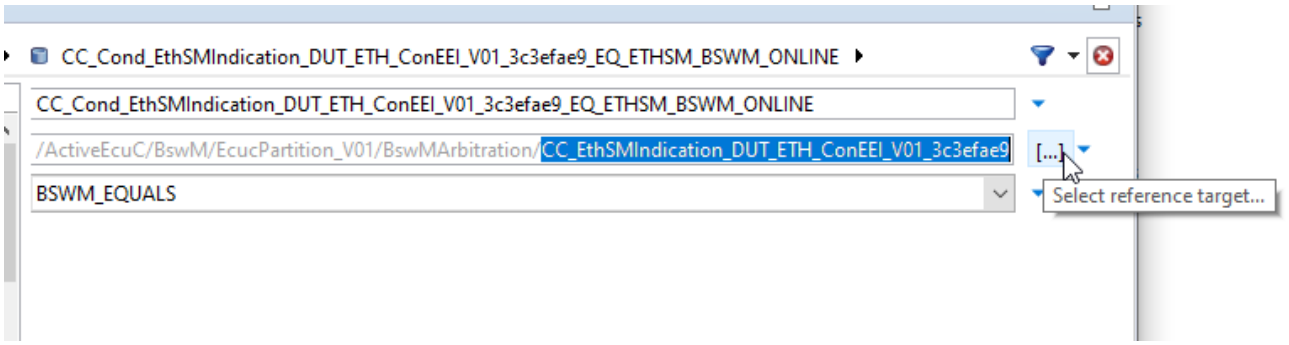


Figure 5-7 BswM Configuration – Correct References

5.1.4 Request Processing

The request processing of the affected BSW Mode Request Ports must be set to BSWM_DEFERRED.

This means, that the request gets processed in the next main function cycle. Immediate processing is not supported for cross-partition use cases.

5.2 Can

Certain Can driver support the BSW-Split Multi-Instance ETH use case. In this use case, the communication of the existing Can controller is processed by specific Partitions. The actual partition mapping is done in CanIf. Details on the configuration can be found in the technical references of CanIf ([12]) and Can ([27]).

5.3 CanIf

CanIf provides support for the BSW-Split Multi-Instance CAN use case. Details on the configuration can be found in the technical reference of CanIf ([12]).

5.4 CanTp

CanTP provides support for the BSW-Split Multi-Instance CAN use case. Details on the configuration can be found in the technical reference of CanTP ([28]).

5.5 CanTSyn

CanTSyn provides support for the BSW-Split Multi-Instance CAN use case. Details on the configuration can be found in the technical reference of CanTSyn ([13]).

5.6 Com

Com allows the configuration of multiple main functions for transmission and reception. This feature allows to split-up the overall computation time of the Com main functions. The following split-up scenarios are supported:

> Multiple main functions within one OS Application. E.g. to process signals with the same cycle-time or signals of different field busses in dedicated main functions

> Multiple main functions on multiple cores.

Split-up means that a Com main function processes a configurable set of PDUs and the therein contained Signals. The Rte will create efficient direct Com accesses to signals from/to application SWC within the same partition. Signals of PDUs that are processed by main functions on other partitions are accessed over core boundaries via Rte-internal mechanisms.

Configuring main functions on several cores is especially effective in combination with BSW-Split. The PDUs of one bus can be assigned to main functions on the same partition and hence do not need to cross partition/core boundaries.

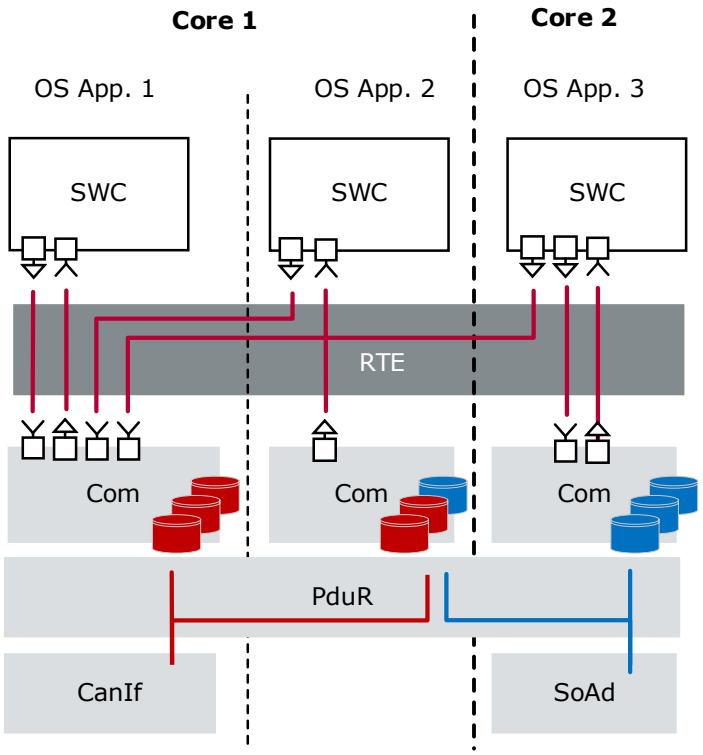


Figure 5-8 Com General Architecture

5.6.1 Com Main Function Mapping

Com allows to configure multiple `MainFunctionRx` and `-Tx` as shown in Figure 5-9. The main functions must be mapped to ECUC partitions via the parameters:

> `ComMainRxPartitionRef`

> `ComMainTxPartitionRef`

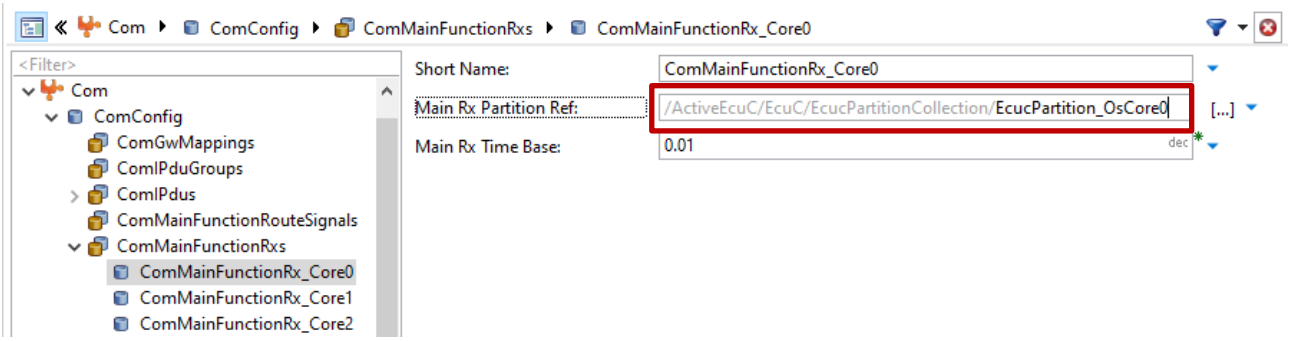


Figure 5-9 Com Main Function Partition Mapping

The PduR will automatically route the PDUs between Com and bus interface.

5.6.2 Com PDU – Main Function Mapping

The individual PDUs must be mapped to the desired main functions as shown in Figure 5-10.

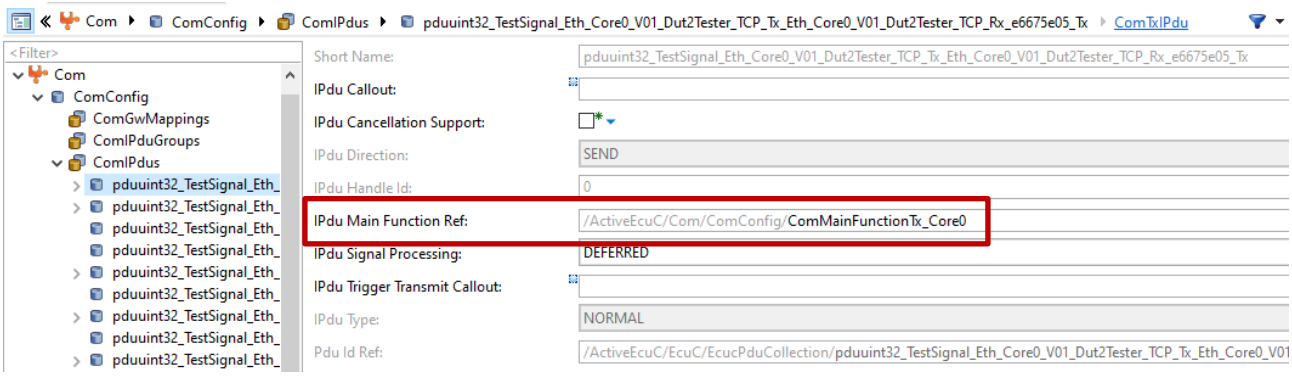


Figure 5-10 Com PDU Main Function Mapping

> The corresponding parameter is: `ComIPduMainFunctionRef`.

Note
The PduR configuration may need to be adjusted/completed after PDU – Main Function assignment in Com. Validation warnings/error show the required actions.

Please also refer to the technical reference ([9]) of the Com for more information on this topic.

5.7 ComM

ComM supports the BSW-Split use case by allowing to map ComM channels to ECUC partitions.

> The configuration is simple. ComM provides the parameter `ComMChannelPartitionRef`. It needs to be set to the partition, to which this channel is mapped to (please refer to chapter 6 for more details).

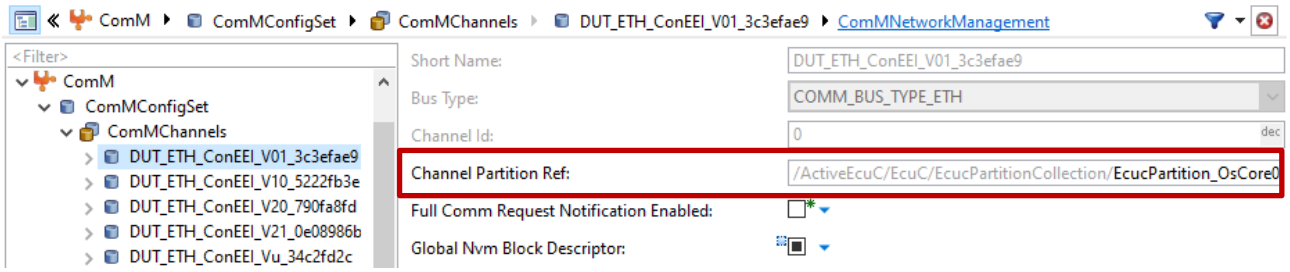


Figure 5-11 ComM Channel Mapping

> ComM has a master partition, on which also the service SWC is available. It is configured via the parameter `ComMEcucPartitionRef`. For each channel that is mapped to a different partition, ComM will create an additional satellite main function which is named in accordance with the partition it belongs to:

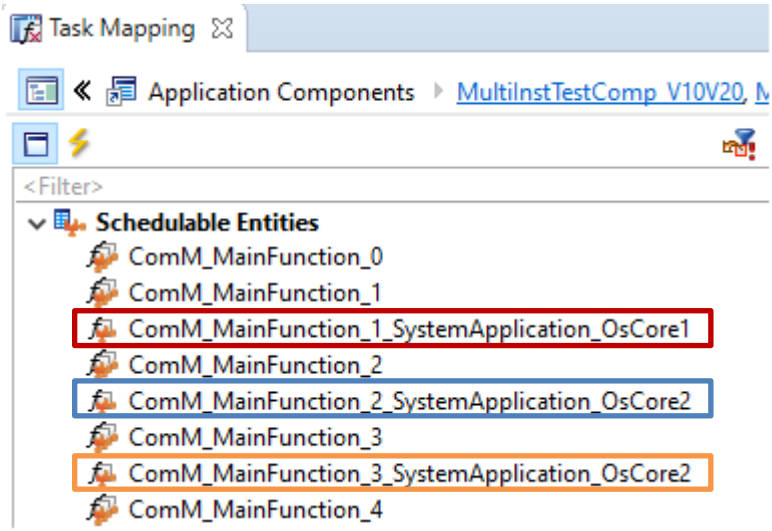


Figure 5-12 ComM Main Functions (Example)

Those main functions must be mapped to tasks that are assigned to the corresponding OS Application.

Note
ComM main functions without a partition-suffix must be mapped to a task of the ComM master partition (`ComMEcucPartitionRef`).

5.8 Dem

The multi-partition feature of the Dem provides the most frequently called service interfaces `Dem_GetEvent()` and `Dem_SetEvent()` for the application SWCs in the same Partitions as the application SWC that uses those services. As a result, the invocation of those services is always realized as simple function calls with virtually no overhead within the Rte. Extensive information on Dem's features can be found in the technical reference of Dem ([1]).

To use this feature, a couple of additional configuration steps need to be done:

1 First, the Dem events (`DemEventParameters`) need to be assigned to OS Applications via the following parameter:

2 `DemEventOsApplicationRef`: This reference determines the Dem satellite and therefore on which partition this event is monitored. Per default this reference is not set which means that the event is handled by the Dem satellite that resides on the same OS Application as the Dem master. If an OS Application is referenced for the first time, a Dem satellite on that partition is generated automatically.

3 Using the Dem monitor interface for an event mapped to a different OS Application will result in the implementation of an RPC. Furthermore, in the general settings the following two parameters must be configured: `DemMasterOsApplicationRef`: OS Application on which the Dem master component is assigned to. `DemOsApplicationRef`: In this list, all OS Applications which execute a Dem component (master or satellite) must be listed.

Note
There is an auto solving action for populating this list.

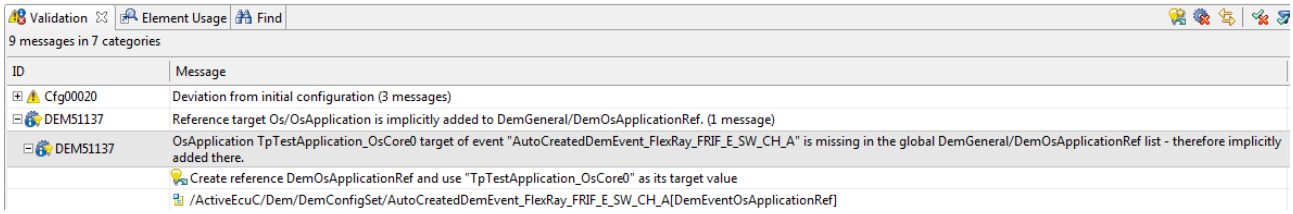


Figure 5-13 Auto Solving Action DemOsApplicationRef

5.8.1 Dem Task Mapping

For the Dem master and each Dem satellite, the cyclic main functions need to be mapped to an appropriate BSW task. The task must belong to the OS application, which is indicated by the satellite's name.

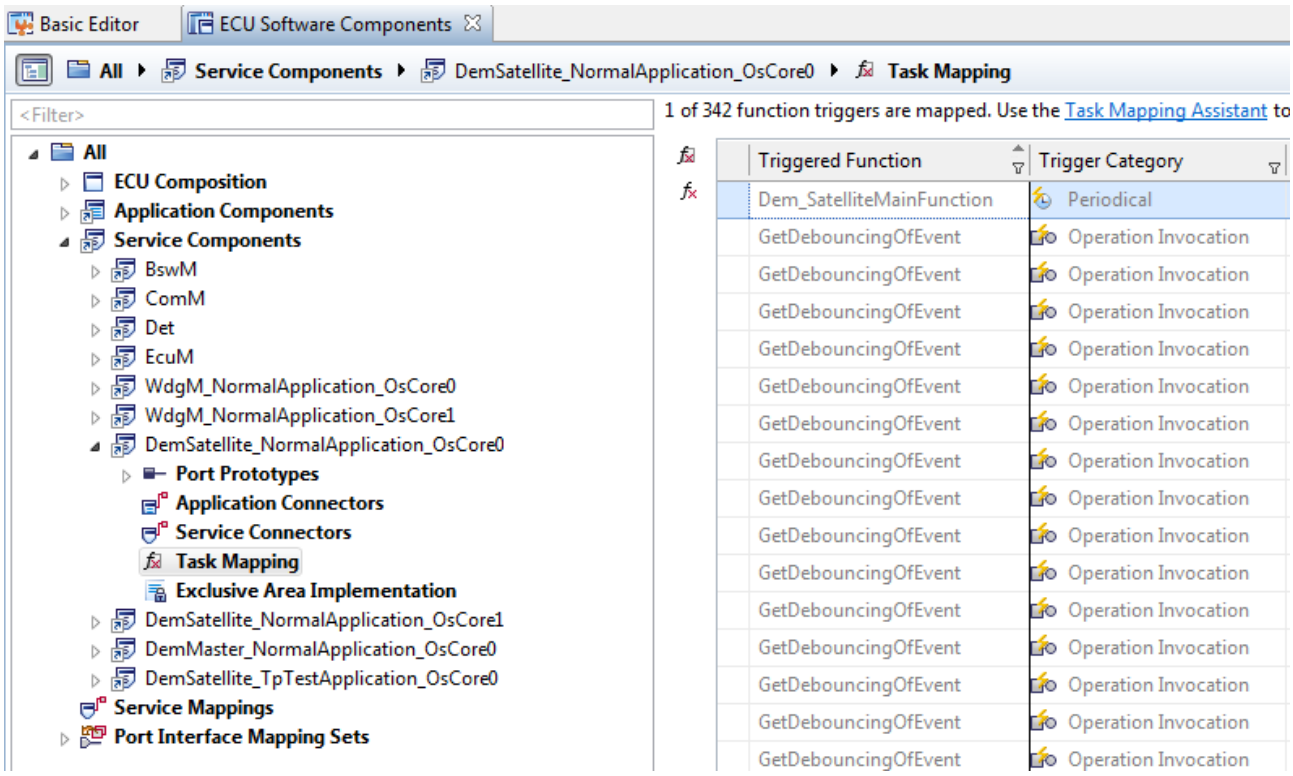


Figure 5-14 Dem Task Mapping

5.8.2 Dem Data Consistency

Note
Please make sure to check chapter Synchronization in the technical reference of Dem.

5.8.3 Det

Det has no multi-partition specific configuration parameters.

5.8.4 Det Data Consistency

Note
Please make sure to check chapter **Multi-core system** in the technical reference of Det.

5.9 EcuM

The Os has a comprehensive technical reference ([11]) which should be considered as first source of information.

4 EcuM provides one instance per core that needs to be managed. Essentially that applies to all cores that are configured as `OsCoreIsAutosar` in the OS. (Compare to chapter 5.16)

5 From the EcuM's perspective, there is one master core and one or multiple slave cores. Slave-core handling is enabled per core by creating a `EcuMPartitionRef` which references a `EcucPartition` in /EcuM/EcuMConfiguration/EcuMFlexConfiguration/:

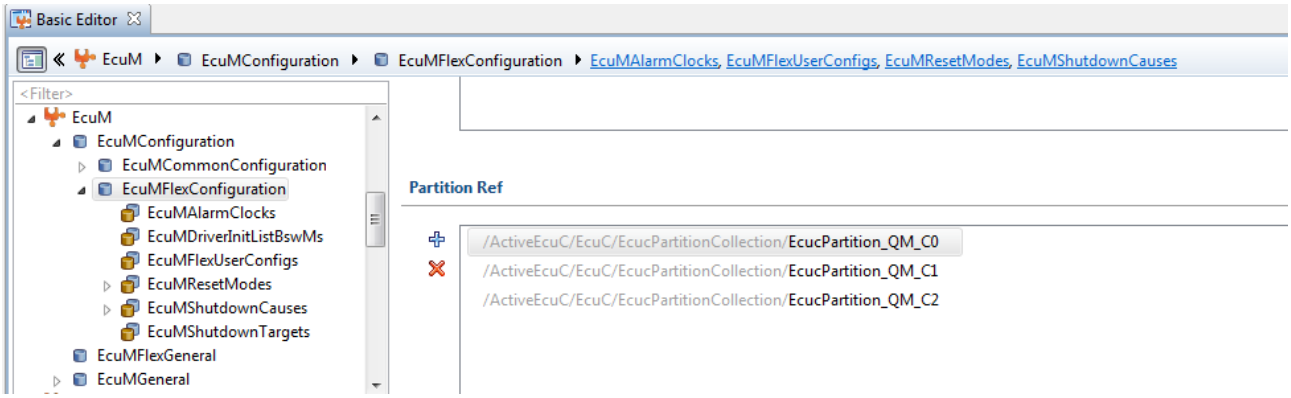


Figure 5-15 EcuM Partition References for Slave Core Handling

5.9.1 EcuM Task Mapping

The EcuM provides one common main function, which must be called on each core it handles. This is done by mapping the schedulable entities of the EcuM (one per core) to appropriate tasks in the Task Mapping editor.

5.10 IpduM

The IpduM can perform PDU multiplexing on multiple partitions. This has the decisive advantage that PDUs can be processed in the same partition as they are received or transmitted. Additional partition transitions of PDUs is not necessary.

IpduM provides main functions for each partition on which it performs multiplexing. Those main functions must be mapped to tasks in the corresponding partition. Also, the contained and container PDUs must be mapped to this partition.

Further configuration is not required.

5.11 Eth

Certain Eth driver support the BSW-Split Multi-Instance ETH use case. In this use case, the existing Rx/Tx queues are processed by specific partitions.

Multi-Instance ETH requires an Eth Vlan ID Based Frame Queuing Configuration (`EthVlanIDBasedFrameQueuingConfig`). Queues get assigned to cores. It is hence project specific how many queues are needed and how the mapping of VLANs to queues (and then effectively cores) looks like.

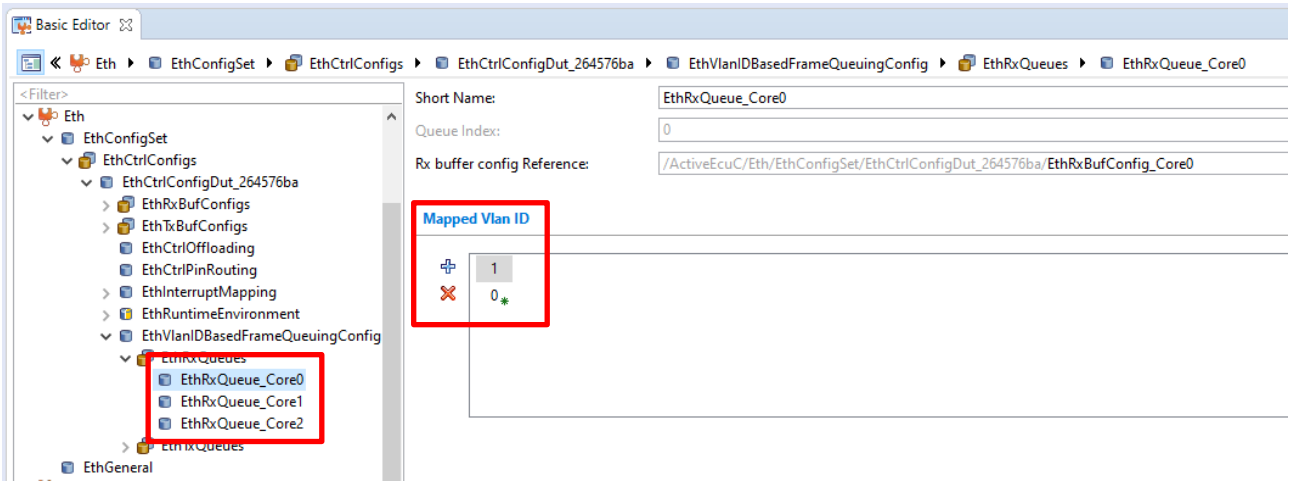


Figure 5-16 Eth – VLAN Queue Configuration

5.11.1 ISR OS Application Mapping

Effective mapping of queue processing to a core is achieved by mapping interrupt service routines to the corresponding OS Applications.

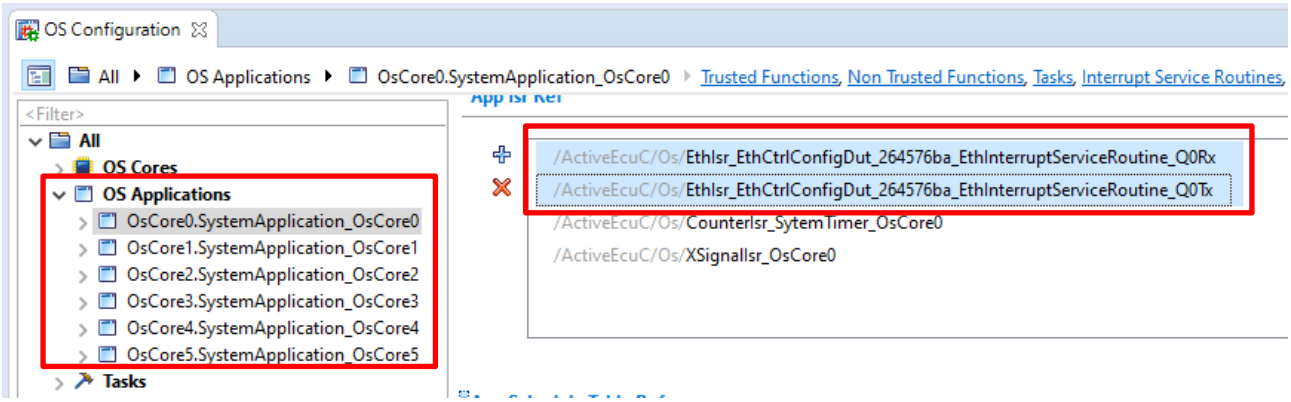


Table 5-1 Eth – ISR – OS App Mapping

Note
BSW-Split Multi-Instance ETH only supports interrupt driven processing.

5.12 EthIf

EthIf supports the BSW-Split Multi-Instance ETH use case. In this use case, the existing EthIf Controllers are processed by specific partitions.

Further information on EthIf's multi-partition features is given in its technical reference ([14])

Multi-Instance ETH requires mapping of the EthIf Controllers (`EthIfController`) to EcuC partitions. This is done by configuration of the reference parameters: `EthIfEcucPartitionRef`.

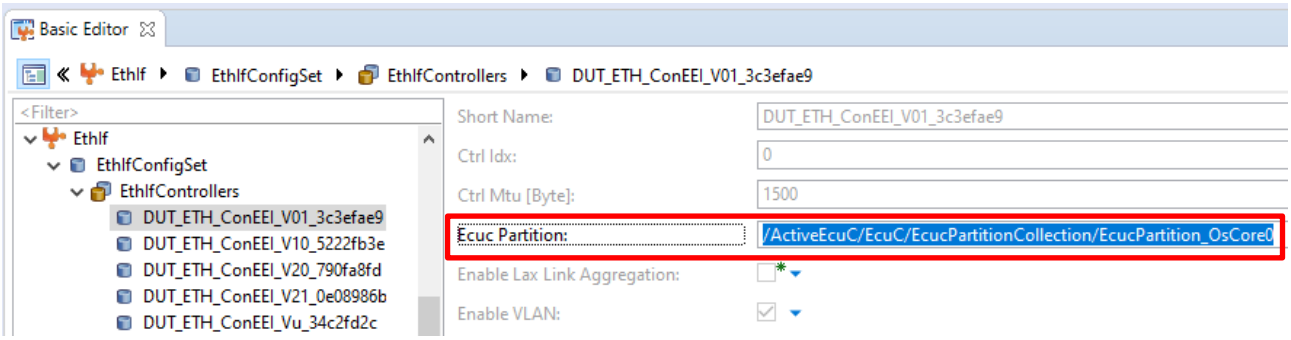


Figure 5-17 EthIf – Partition Mapping

Note
The Partition / OS Application mapping of Eth and EthIf must match. The EthIf Controller and the interrupt of the Eth queue of the same VLAN must be mapped to the same OS Application / ECUC Partition.

5.13 FiM

The multi-partition FiM has satellites that can provide the `FunctionInhibition` interface in the same partitions as the application SWC, that use this service, are mapped to. This means that the invocation of `FiM_GetFunctionPermission()` by the application is always implemented as simple function call in the Rte and has no overhead due to multi-partitioning.

To make use of this functionality, a couple of additional configuration steps need to be done:

1 The FiMFIDs need to be assigned to FiM satellites and thus OS Applications via the following parameter:

> **FiMSatelliteRef**: This reference determines the FiM satellite that performs the processing of the monitor status changes of the Dem satellite in the same partition. Configuration of this parameter is mandatory.

2 Further configuration is required in the FiM config set:

> **FiMSatellite**: Configuration of the FiM satellites that will be created. There must be one FiMSatellite container for every OS Application in which the FiM shall be present in. FiMSatellites are mapped to OS Applications by setting the reference parameter: **FiMOsApplicationRef**.

3 In FiM general, the FiM master must be assigned to an OS Application:

> **FiMMasterOsApplicationRef**: OS Application on which the FiM master component is being executed. This must be the same OS Application as configured for the Dem master.

5.13.1 FiM Task Mapping

Task mapping is only required if the following parameter is set to true:

> FiMCyclicFidCalculation

If this parameter is set to “TRUE”, the cyclic main function of the FiM master (`FiMMaster_<Os Application>_MainFunction`) must be mapped to a task on the OS Application to which the FiM master is assigned to.

To propagate the OS Application assignment of the FiM to the Rte, the FiM satellites and the master service SWC are mapped to ECUC partitions.

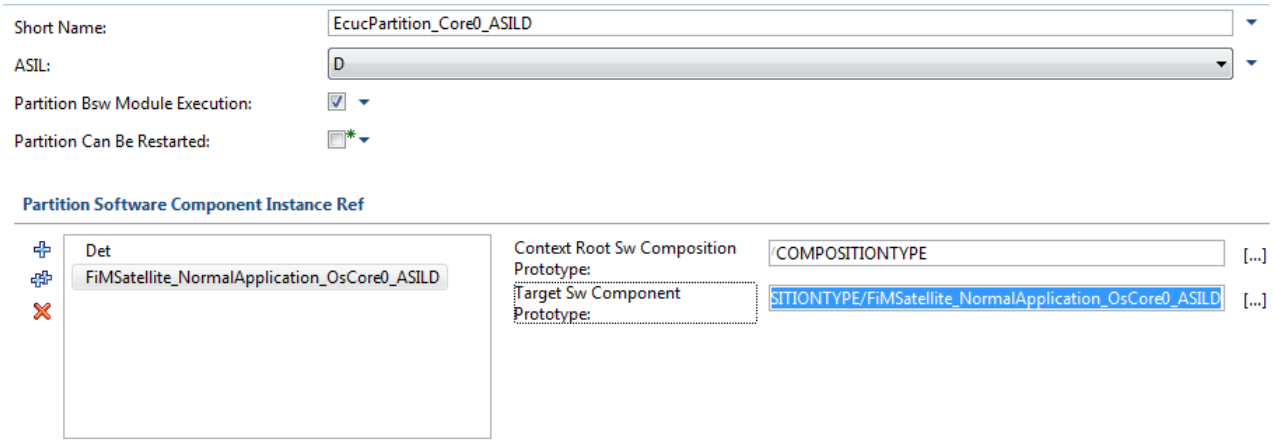


Figure 5-18 FiM Partition Assignment

For more information on OS Application/Partition mapping please refer to chapter 5.25.1.

5.13.2 FiM Data Consistency

Note
Please make sure to check chapter Synchronization in the technical reference of FiM.

5.14 LdCom

LdCom supports PDU provisioning from- and to the application SWC on arbitrary partitions. The partition is determined by the parameter `EcucPduDefaultPartitionRef` respectively `EcucPduDedicatedPartition` of the respective ECUC global PDUs. Further configuration steps with respect to multi-partitioning are not required.

More information on the LdCom configuration can be found in its Technical Reference.

5.15 Nm

The network management interface supports the BSW-Split use case and allows channels on multiple partitions. Nm and ComM configuration are closely related since Nm channels have references to ComM channels. This allows Nm to automatically detect, on which partition an Nm satellite is needed.

Configuration tasks are:

> Configure ComM first (compare to chapter 5.7)

- Configure the Nm master partition by setting **NmEcucPartitionRef**
- Mapping of the Nm main functions
  - **Nm_MainFunction**: Main function of the master. Must be mapped to a task that belongs to the same partition as referenced by `NmEcucPartitionRef`
  - **Nm_MainFunction_<ECUC-Partition>:** Satellite main function. Must be mapped to a task that belongs to the same partition as indicated by the name suffix of this main function (<ECUC-Partition>)

5.16 Os

The Os has a comprehensive technical reference ([4]) which should be considered as first source of information.

- **OsCoreIsAutosar**: Only if this check box is activated, a core can be used to execute AUTOSAR BSW or application SWC.
- **OsCoreIsAutostart**: This must only be activated at cores which start up automatically after power on. Typically, this option is activated on only one core. If, however the master/slave core behavior is emulated in software, this option is activated on all cores which start up automatically.
- **OsCoreXSignalChannel**: Xsignal channels are required for the Os' internal cross-core communication. For example, to set an alarm or activate a task on another core. There is an auto solving action to create the required XsignalChannels. If the recommended channels are not created, the Os cannot work properly.
- **OsCoreXSignalChannelSize**: Determines how many Xsignal events can be queued for this channel. The exact number highly depends on the system configuration. A number between five and ten is however a good starting point. The Os will report an error if the queue overflowed. So, it is possible to approach the required queue size bottom up.
- **OsSpinlockLockMethod**: Minimizing the time a core possesses a spinlock is paramount for efficient multi-core systems. Pure spinlocks do however not affect the core-local scheduling of the AUTOSAR Os. Hence it is easily possible for tasks to get preempted while holding a spinlock. To prevent this unwanted behavior, spinlocks can be combined with a locking method which affects/prevents the core-local scheduling.
- **OsSpinlockLockType**: Minimizes the execution time of the spinlock API if set to OPTIMIZED. This is achieved by omitting the API checks specified by AUTOSAR (deadlock, nesting etc.) and keeping the spinlock data in user memory which eliminates the need for an Os context change. However, this only applies if all accessing OS Applications have the same trusted-setting.

5.17 PduR

PduR provides support for the BSW-Split use case and allows both:

- Routing of PDUs between different partitions and
- Routing of PDUs within one partition on multiple partitions simultaneously

Details on the configuration can be found in the technical reference of PduR ([12]). A couple of parameters are explained in the following:

> **PduRSupportMulticore**: This check box enables the general multi-core support of the PduR. It must be enabled in BSW-Split configurations.

> **PduREnableDeferredReception:** Must be enabled if the relocated bus receives PDUs which are routed to the BSW core.

> **PduREnableDeferredTransmission:** Must be enabled if the relocated bus transmits PDUs which origin from the BSW core.

> **PduRMainFunctionPeriod:** Cycle time of the PduR main function. Lower values result in less delay during deferred reception/transmission but at the same time to more execution time overhead.

> **PduRMulticoreRoutingPath:** Determines if the routing path is inter-core or intra-core. Must be enabled for inter core routings.

5.18 StbM

StbM offers its service interfaces towards the application software via service SW-C. StbM can offer multiple service SW-Cs on different partitions. A service SW-C is offered on those partitions that are referenced by the time bases:

> **/MICROSAR/StbM/StbMSynchronizedTimeBase/StbMOsApplicationRef**

StbM uses main functions for time base handling. One main function can handle one or multiple time bases. If the StbM generates a main function for a partition depends on the type of time base (master/slave) and the partition mapping of the accessing application:

1. A **slave time base** is handled by an automatically created StbM main function on the same partition as the associated <bus>Tsyn
2. A **master time base** is handled by an automatically generated StbM main function on the same partition on which the `GlobalTime_Master` service interface is connected to a SW-C (Port mapping) (→ This automatically means, that only one SW-C may connect to the `GlobalTime_Master` interface of a time base)



Note
Using the StbM services by the application SW-C does not require a StbM main function on this partition.

The following Figure 5-19 shows how the mapping of the time bases works:

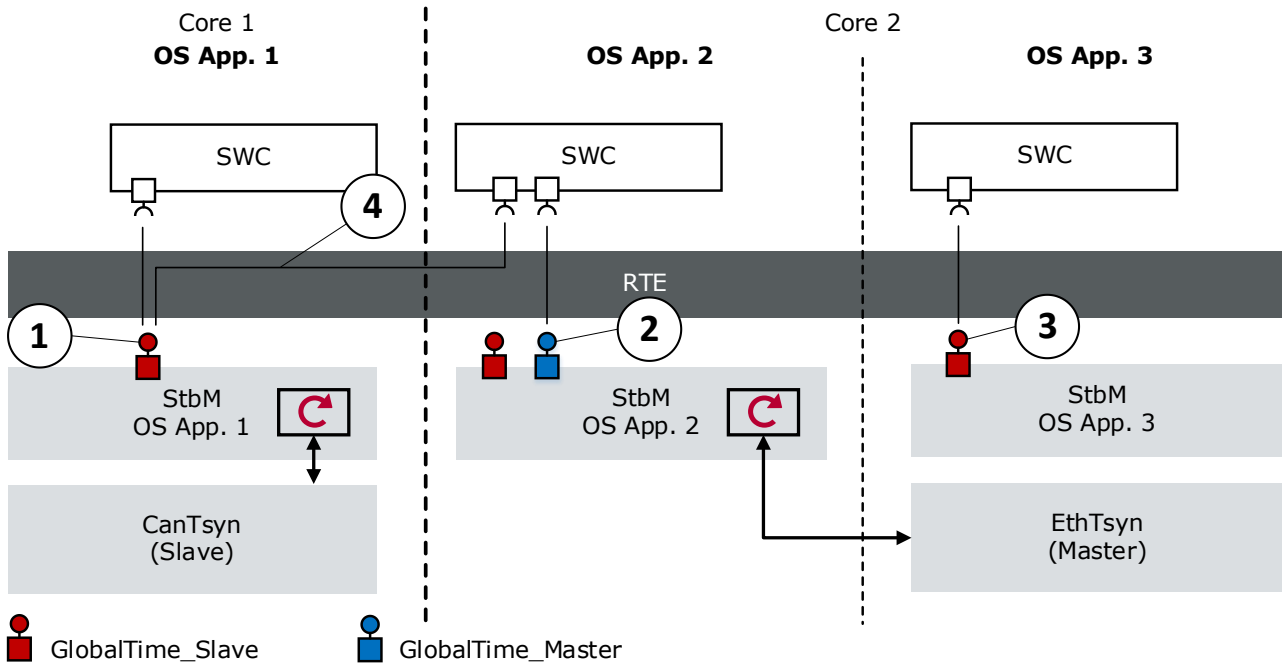


Figure 5-19 StbM Partition Mapping

1. StbM handles the slave time base with a main function on the same partition. The application SW-C efficiently connects to a slave port on the same partition.
2. The application SW-C maps the master time base to this partition by connecting to the master port. StbM provides a main function for this partition. The associated <bus>Tsyn may be mapped to different partitions.
3. The application SW-C efficiently connects to a slave port on a different partition. The Partition crossing is done by the StbM.
4. The application SW-C connects to the slave port provided by the StbM on a different partition. The partition crossing is done by the RTE.

Note
When connecting a SW-C on a different partition to a StbM port (Item “3” in Figure 5-19), the SIL of the time base degrades to the smallest level of all involved partitions.

Please refer to the technical reference of the StbM for additional information regarding configuration and usage.

5.18.1 StbM Data Consistency



Note
Please make sure to check chapter **Critical Sections** in the technical reference of StbM.

5.19 SoAd

The Socket Adaptor supports the BSW-Split use case and allows mapping of data paths on multiple partitions.

Detailed description on the functionality and the configuration can be found in the technical reference of SoAd ([12]).

5.19.1 SoAd Data Consistency



Note
Please make sure to check chapter **Critical Sections** in the technical reference of SoAd.

5.20 TcpIp

TcpIp supports the BSW-Split use case and allows mapping of TcpIp controllers on multiple partitions.

Detailed description on the functionality and the configuration can be found in the technical reference of TcpIp ([12]).

5.20.1 TcpIp Data Consistency



Note
Please make sure to check chapter **Critical Sections** in the technical reference of TcpIp.

5.21 WdgIf

WdgIf offers features for multi-core use cases. It can either handle different physical watchdog devices on different cores or combine the states of multiple WdgM instances on different partitions while handling one physical watchdog.

An extensive description of the features and configuration of WdgIf can be found in its technical reference ([12]).

5.21.1 WdgIf Data Consistency



Note
Please make sure to check chapter **Services used by WdgIf** in the technical reference of WdgIf.

5.22 WdgM

WdgM can provide multiple instances, so that watchdog supervision can be done locally within the different partitions. This has the great advantage, that the WdgM Service Interfaces can be accessed with least overhead through the Rte.

An extensive description of the features and configuration of WdgM can be found in its technical reference ([19]).

5.23 veRtm

veRtm supports execution time and CPU load measurements on multiple cores. The supported features and required configuration are described in its technical reference [20].

5.23.1 veRtm Data Consistency



Note
Please make sure to check chapter **Measurement on multi core system** in the technical reference of veRtm.

The requirement to use a Spinlock in multi-core configurations applies to all three Exclusive Areas of Rtm.

5.24 Xcp

Xcp provides a set of multi-core features

- **Type Safe Copy**: To access variables that belong to a partition different to the one, where the XCP Protocol Layer runs on.
- **DAQ/STIM with Multi-Core**: The function XcpEvent can be used on multiple cores to trigger data transmission.
- **XCP Interface Modules on different cores**: The main function of Xcp can be mapped to a different core than the main functions of the underlying <bus>Xcp. (Note: The PDUs must be mapped the partitions of the corresponding <bus>XCP)

Description of the features and configuration can be found in technical reference of Xcp ([21]).

5.25 Partition Mapping of SWC / Service SWC

Partition assignment of SWC determines how the Rte generates the interface connections from and to those SWC. The Rte produces different implementations depending on the partition assignment of the source/client and sink/server of an interface.

In many cases, the Rte can determine the partition assignment of the SWCs automatically by its task mapping. However, there can be SWCs which only contain runnable trigger for which task mapping is optional. A typical example for that are server runnables. In such cases, the integrator needs to supplement the configuration by this information. There are two variants to do so:

- Mapping of the SWC to a partition (recommended since it causes less execution time overhead)
- Mapping of the server operation to a task (and thus indirectly to a partition).

The following chapters describe both variants.

5.25.1 SWC Mapping

This variant explicitly links a SWC to a partition. Doing so, gives the Rte the knowledge, that all unmapped runnables of this SWC belong to the referenced partition.

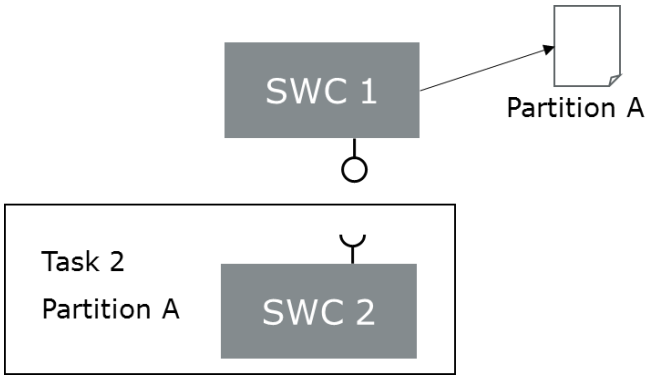


Figure 5-20 SWC Mapping

Partition mapping is done in the EcuC module in the generic editor. SWC that shall be assigned to a partition must be added in the Partition Software Component Instance Ref.

Note
If caller and callee are mapped to different partitions, a task mapping for the affected server runnable is required anyway. However, if this is not the case, a direct synchronous call is yielding optimal performance.

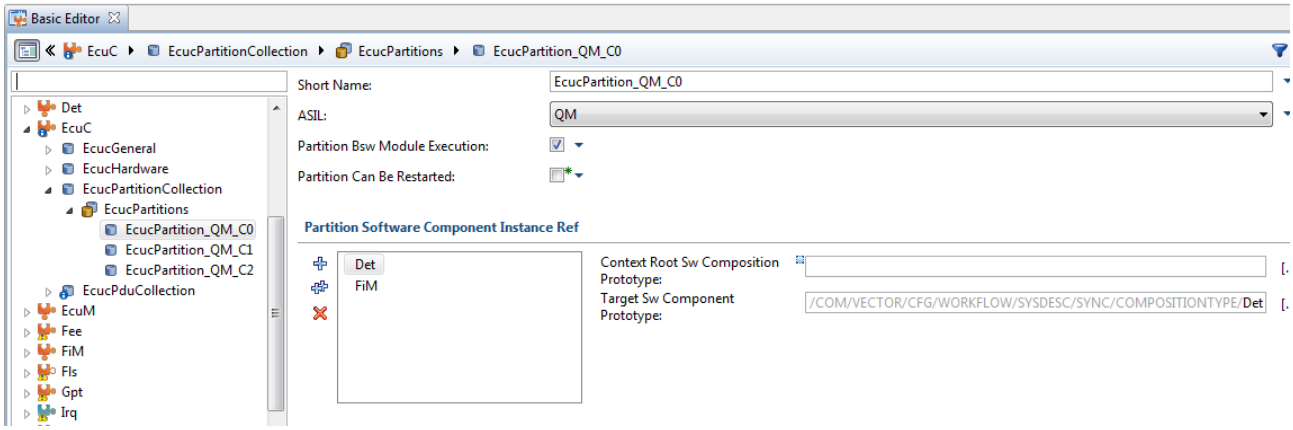


Figure 5-21 EcuC Partition Mapping

5.25.2 Task Mapping

In this variant, one or all server runnables are mapped to a task which in turn is assigned to a partition.

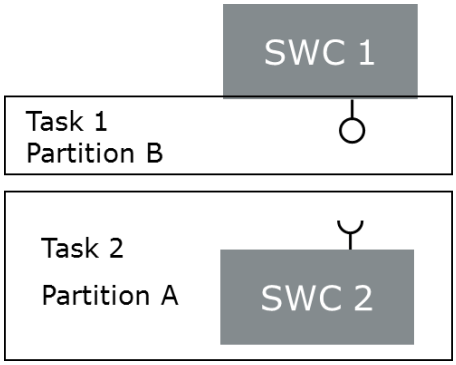


Figure 5-22 Task Mapping

The big downside of this approach is that the operation is always executed in the context of the task to which it is mapped to. In many cases, this means that the Rte implements a remote procedure call (RPC) to execute the operation. This requires considerably more execution time than a plain function call (the standard implementation if caller and callee are mapped to the same partition).

Runnable mapping is done in the Task Mapping Assistant.

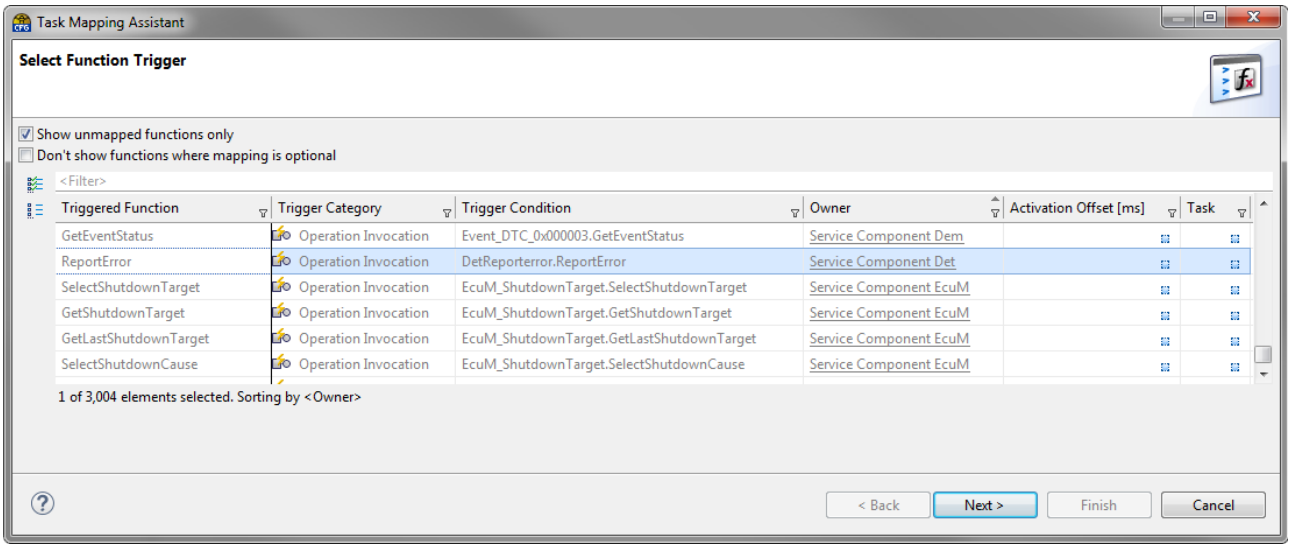


Figure 5-23 Task Mapping Assistant

Note
To avoid exceedingly long waiting times for RPC execution, server runnables need to be mapped to tasks with high priority.

5.26 Rte/SchM Runnable Timing

During startup, SchM and Rte activate the Os Alarms that trigger execution of the App SWC and BSW runnables. In the default configuration, it is not unlikely, that the alarm activation is spread over a longer time span and, as consequence, runnables of the Rte and SchM unintendedly get activated at different activation points.

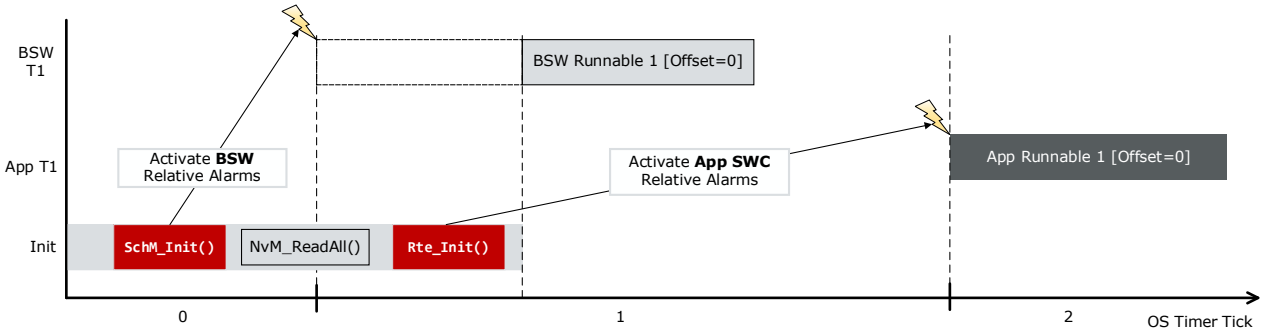


Figure 5-24 Rte/SchM Runnable Timing – Standard Behavior

Figure 5-24 shows, how the init-task (often due to `NvM_ReadAll()`) shifts the activation point of **App Runnable 1** to the next tick of the Os timer. App and SchM runnables will maintain this displacement by one timer tick indefinitely. This causes additional Os overhead and a different timing behavior than expected.

To reduce the risk of SchM – Rte runnable displacement, the **Start Timing** features of EcuM and Rte can be used. When activated, both components will provide an API that is dedicated to activate the Os Alarms. Both APIs can be called in short succession, thus minimizing (but

not eliminating) the risk that SchM and Rte alarms get activated during different values of the Os timer tick.

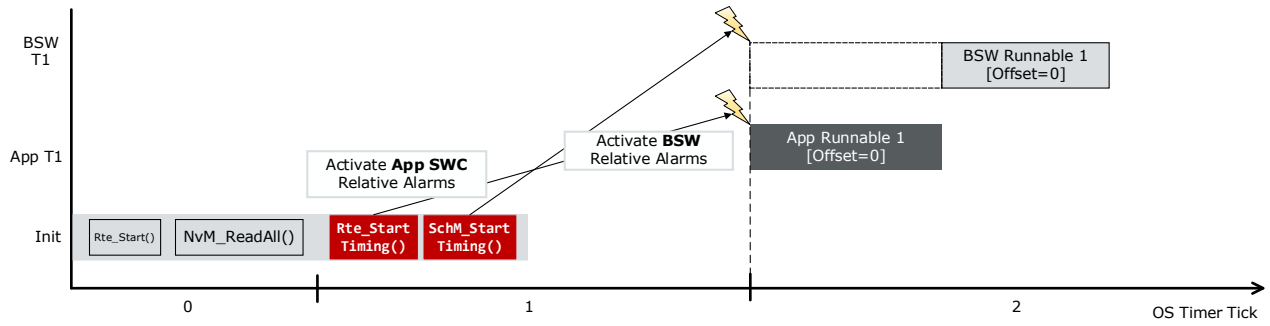


Figure 5-25 Rte/SchM Runnable Timing – Using Rte/SchM_StartTiming

To activate this feature:

> Set the following parameter to TRUE:
`/MICROSAR/EcuM/EcuMGeneral/EcuMSchMStartInitialization`

> Create a Rte Initialization Runnable Batch:
`/MICROSAR/Rte/RteInitializationRunnableBatch`

> Execute the **Auto Configuration: Module Initialization** with **Rte_StartTiming** checked:

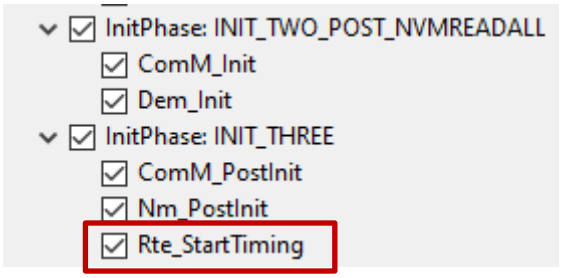


Figure 5-26 Rte/SchM Runnable Timing – Auto Configuration: Module Initialization

6 BSW-Split

The feature BSW-Split allows to map communication clusters (Ethernet, CAN, FlexRay, Lin) to partitions with the goals of:

- CPU load balancing
- Safety separation

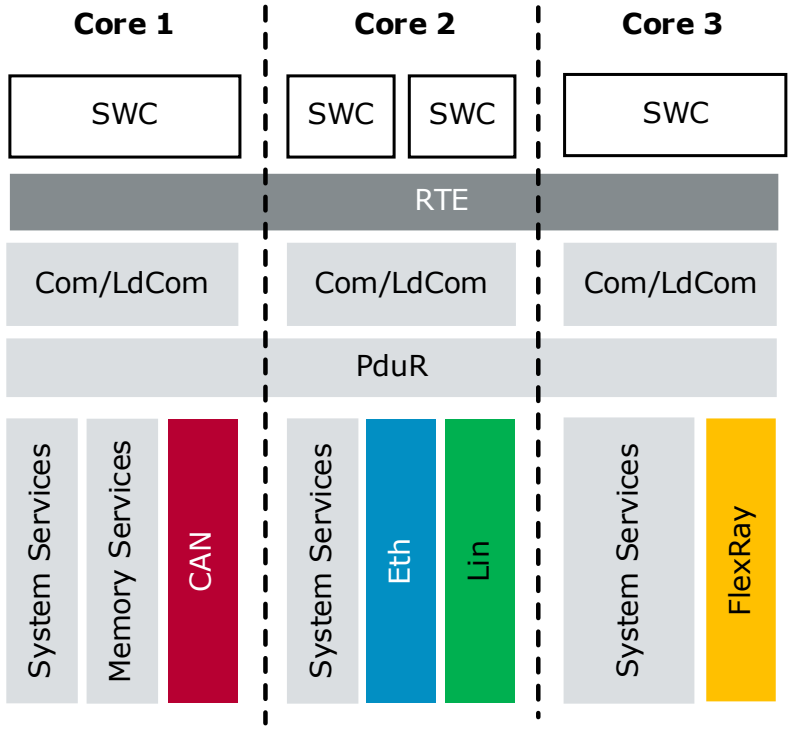


Figure 6-1 BSW-Split Overview

The mapping of individual parts of the clusters is supported for CAN and ETH on specific microcontrollers. Here, the mappable elements are:

- CAN channel
- ETH VLANs

More to this topic can be found in chapter 7.

Mapping of a communication cluster to a partition means, that the execution of the cyclic main functions and the interrupts of such a cluster takes place in the context of this partition- respectively core. By distributing the communication clusters to different partitions on different cores, the CPU load of the communication stack can be balanced and/or the BSW stack can be distributed to different safety partitions.

This load balancing is an additional degree of freedom for the multi-core software distribution to the application SWC mapping.

However, in contrast to application software distribution, this feature is more complex and has some drawbacks. Foremost these are:

- It does **not** increase the processing speed of the BSW (multi-core speedup) since the processing tasks are merely relocated but not parallelized. Hence, processing will even in an ideal scenario take at least the same time like on single-core systems. Realistically, it takes even longer since additional program logic (overhead) is required to exchange the data and control flow between the cores.
- It does **not** improve efficiency of the BSW stack. Several interfaces need to be called across core boundaries and hence require a more complex implementation with greater execution time overhead. The BSW stack has least overall execution time when it is being executed on only one core.
- More memory is required. The PduR buffers all PDUs which are routed between the cores to ensure data consistency and availability. This may not be required in single core configurations.
- The extent of the load distribution depends on many factors, hence reliable estimations cannot be given. Main influencing factors are
  - Bus type
  - Amount and type of the PDU routings



Note
The best load distribution can be expected when bus systems are relocated, which cause considerable execution time in their main functions and interrupts and which transport mostly unbuffered gateway PDUs and forwardings with direct data provision.

As consequence, the data provision should be changed to “Direct” whenever possible.

6.1 Architecture

Main element of the BSW-Split feature is the enhanced PDU router. It is able to route between upper- and lower layer modules on different cores.

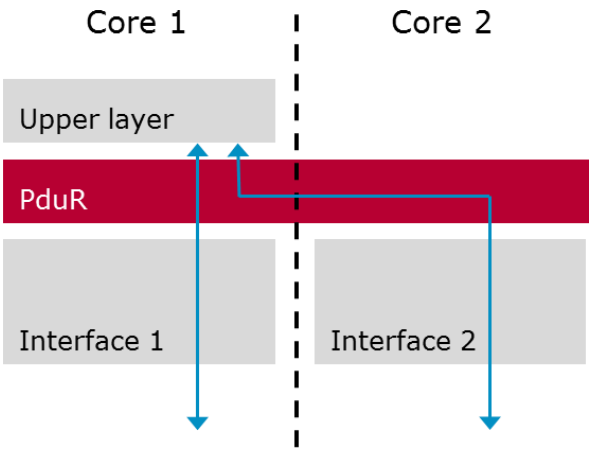


Figure 6-2 BSW-Split Vertical Cut Architecture

This multi-core routing can be applied to the following elements for the interface and transport protocol APIs:

- Transmit requests (PDUs)
- Rx Indications
- Tx Confirmations

Each core handles one or more communication interfaces. If several busses are mapped to the same core, routing between these busses is performed core local.

The data exchange of the routing is done via two fundamental different approaches.

- IF API forwardings and unbuffered routings are handled by universal, unidirectional and Spinlock-free queues. The queues buffer PDUs, Rx indications and Tx confirmations in a FiFo manner. The reception is handled in the context of the PduR's cyclic main function.
- TP API forwardings, buffered routings (TP and IF) and routings with trigger transmit data provision are handled by the standard PduR buffer mechanisms. The PduR employs spinlocks to ensure data consistency.

In both cases, all inter-core routings are handled **deferred** which means that on the receiver core they are processed in the context of the PduR main function.

Non-PDU related interfaces between BSW modules on the different cores are handled depending on their multi-core capabilities. Currently, the majority of the affected modules is multi-core capable. For the following modules, workarounds are required:

- NvM
- Csm
- SPI
- 3rd party drivers
- User developed CDDs

6.2 General Steps

This chapter describes the general steps to relocate a bus cluster from a source OS Application (→ Core) to a target OS Application.

Precondition is, that the general multi-core configuration has already been done and the OS, Rte, EcuM are already running on the target OS Application. For the actual relocation, the following steps must be done:

- Creation of BSW tasks on the target OS Application, that will execute the bus cluster's main functions
- Mapping of the bus cluster's main functions to the new tasks
- Mapping of the interrupt service routines to the target OS Application
- (Optional: Creation of additional Com instances (Rx, Tx main functions) to split up Com's workload)

- Adjust/supplement configurations of the following components
  - PduR
    - Configuration of multi-core in general
    - Configuration of the multi-partition routing paths
  - BswM
    - Creation/extension of a configuration on the target partition (Initialization, Communication Control)
  - ComM
    - Configuration of the multi-core handling
  - Nm
    - Configuration of the multi-core handling
  - StbM
    - Mapping of the time domains (slave/master) to the partitions that have been determined by the time-sync PDUs/Master service interface
  - Com
    - Mapping of the main functions to partitions
    - Mapping of the Com Ipdus to main functions
  - SoAd
    - Maping of the SoAd instance to the target partition
    - Mapping of the Pdus to partitions via the SoAd instance
- Maping of the global PDUs to the partitions of their respective PduR upper- or lower layer module by configuring the `EcucPduDefaultPartitionRef` or in rare cases the `EcucPduDedicatedPartition` parameters
- Identify the need for wrapper components (workarounds) for modules, which APIs are used by multiple partitions
  - NvM
  - Csm
  - SPI
  - 3rd party drivers
  - User developed CDDs
  - …
- Implement integration code
  - Initialization of modules that require multi-partition initialization

- User defined spinlocks for PduR
- Wrapper components (workarounds)

- Adapt memory mapping

6.3 Integration Example: Ethernet

This chapter describes the integration of the BSW-Split feature for a hypothetical configuration. In this example, the Ethernet cluster is relocated from core 0, **EcuCPartition_Core0_ASIL** to the target Partition **EcuCPartition_Core2_ASIL** on core 2. The integration of other bus clusters can be done accordingly.



Caution
The steps and actions shown in this chapter shall merely illustrate the general procedure. They **must** be adapted to the concrete project.

6.3.1 Os

In the Os configuration, the actual relocation of the bus cluster is done. All main functions and interrupt service routines of the ethernet related modules are mapped to an appropriate schedule manager task allocated to the target partition. For this example, the relevant modules are:

- Eth
- EthIf
- EthSM
- EthSwt
- EthTrcv
- Sd
- SoAd
- UdpNm
- TcpIp
- EthTSyn
- DoIP

6.3.2 Com

In this example, the Com configuration gets extended to process the ethernet PDUs in the same Partition. This will be done in a set of new main functions (Rx/Tx):

- Create `Com_MainFunctionTx / Com_MainFunctionRx` for C2 (Ethernet Core)
- Map all main functions to partitions (also the already existing ones) by configuring the references:
  - `ComMainRxPartitionRef`

> `ComMainTxPartitionRef`

> Map all Com Ipdus to MainFunctions by by configuring the reference

> `ComIPduMainFunctionRef`

> Execute solving action to assign the Com Ipdus to the corresponding partitions:

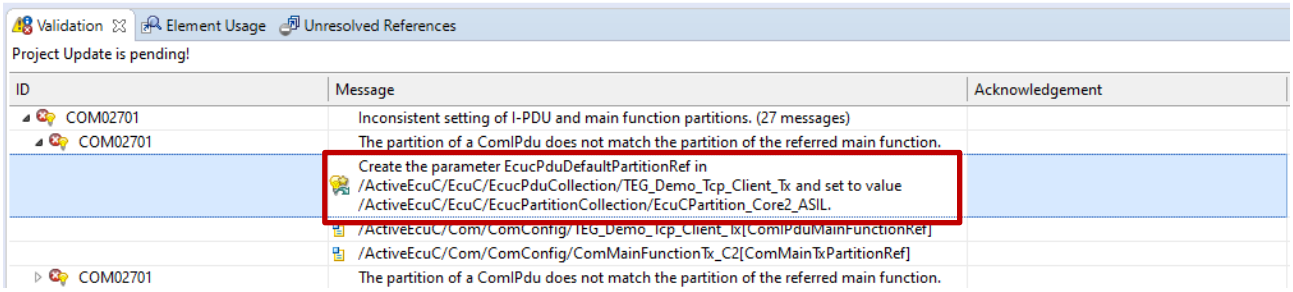


Figure 6-3 Integration Example: Ethernet: Com Ipdu Partition Assignment

6.3.3 SoAd

SoAd already has an instance. However, this instance was most likely not mapped to a partition yet. This needs to be done in this step:

> Reference the target partition in: `SoAdPartitionRef`

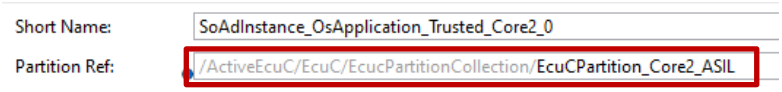


Figure 6-4 Integration Example: Ethernet: SoAd Instance Partition Assignment

6.3.4 CanIf

Although CAN is not relocated, the PduR still needs to know in which partition it runs. This gets configured in this chapter.

> Map the CanIf Controllers to the origin-partition by setting the reference parameter: `CanIfEcucPartitionRef`

> Execute the solving action to map all CanIf PDUs to the origin-partition

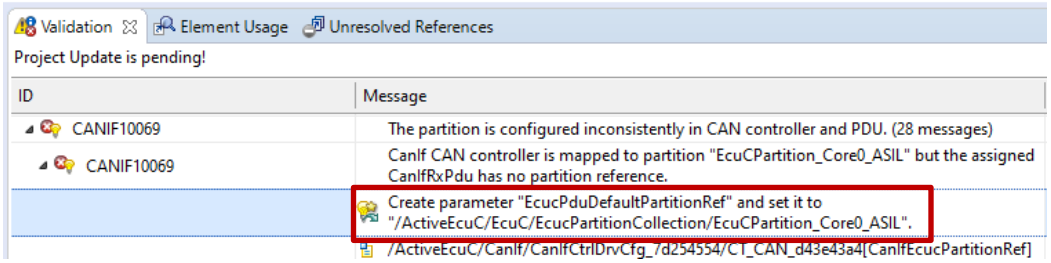


Figure 6-5 Integration Example: Ethernet: CanIf Pdu Partition Assignment

6.3.5 EcuC

The majority of the PDUs should be mapped by now.

> Manually map the remaining EcuC PDUs in accordance with their Bus Cluster or PduR upper layer to the correct partition by setting the `EcucPduDefaultPartitionRef`

6.3.6 PduR

Depending on the configuration, the PduR may or may not need a multi-partition configuration. If there are only routings within the different partitions, no further steps must be taken. However, if there are cross-partition routings, the following steps must be done:

> Set the following parameter to **enabled**: `PduRSupportMulticore`

> Create a multi-core queue by executing the following solving-action:

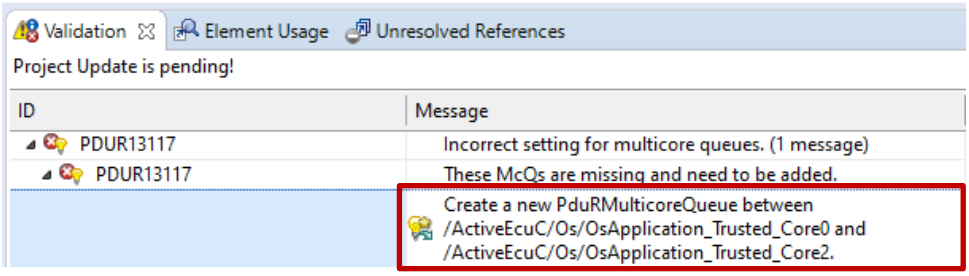


Figure 6-6 Integration Example: Ethernet: PduR Queue Creation

> Configure suitable queue sizes depending on the data rate. E.g.

> `PduRMulticoreQueueSizeBtoA` = 2048

> `PduRMulticoreQueueSizeAtoB` = 2048

> Set the data processing to deferred for all cross-partition routings via the following solving action

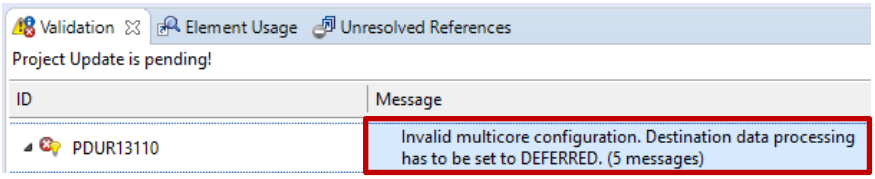


Figure 6-7 Integration Example: Ethernet: PduR Routing Configuration

> Configure a suitable PduR main function period that fits to the project's needs. E.g.

> `PduRMainFunctionPeriod` = 5ms

6.3.7 BswM

In this example, the BswM had no instance on the target partition before the Ethernet relocation. This chapter describes the configuration steps for BswM.

> Create a new BswM instance on the target partition

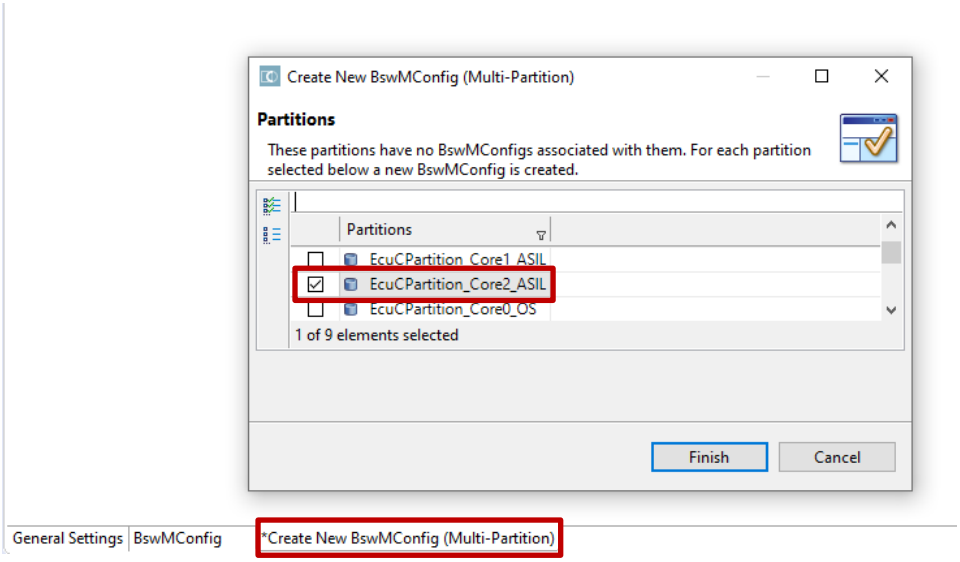


Figure 6-8 Integration Example: Ethernet: BswM Partition Configuration Creation

> Configure the partition references (`BswMPartitionRef`) on all BswM instances

> Execute the **Auto Configuration: Module Initialization** on the original partition

> Remove all Ethernet-related modules on the origin-Partition

> Execute the **Auto Configuration: Module Initialization** on the target-Partition

> Select only the Ethernet-related modules

> Follow the instructions for **Communication Control** described in chapter 5.1.3

6.3.8 ComM

ComM needs to be set up for the multi-partition use case:

> Set the partitions of the ComM Channels appropriately (`ComMChannelPartitionRef`)

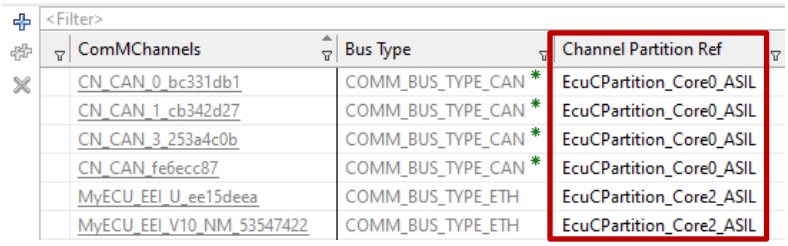


Figure 6-9 Integration Example: Ethernet: ComM Channel Assignment

> Set the ComM Master Partition Ref (`ComMEcucPartitionRef`) to the origin-partition

6.3.9 Nm

Nm needs to be set up for the multi-partition use case:

> Configure the origin-partition as Nm master-partition (`NmEcucPartitionRef`) via the solving action

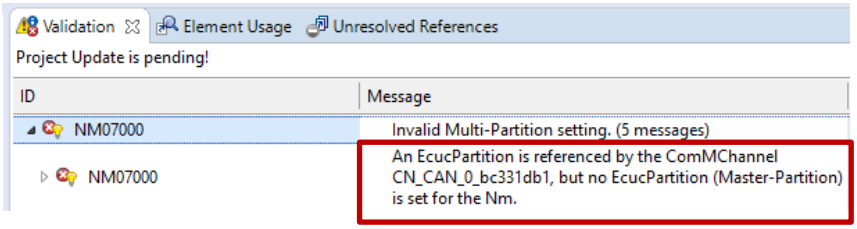


Figure 6-10 Integration Example: Nm Master Partition Assignment

6.3.10 StbM

The StbM time bases must be mapped to partitions. However, this is done implicitly as described in chapter 5.18:

> Map the StbM master/time bases as described in chapter 5.18

> Configure a Spinlock for StbM's exclusive area as described in chapter 5.18.1

6.3.11 Det

Det protects its global data via Exclusive Area 0 that must be protected with a Spinlock:

> Configure the Det Exclusive Area 0 (`RteExclusiveAreaImplMechanism`) to OS_SPINLOCK

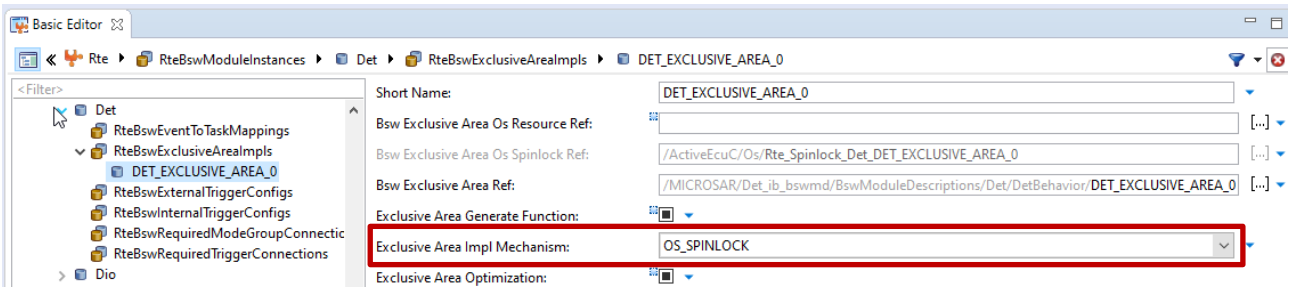


Figure 6-11 Integration Example: Det EA Configuration

The actual Spinlock gets configured and referenced automatically upon Rte generation.

6.3.12 Rte

Finally, a couple of configuration steps must be done in the Rte:

> Set Optimization Mode (`RteOptimizationMode`) to "RUNTIME"

> Map Cylic runnables (main functions) to appropriate Tasks in the correct partitions

6.3.13 Other Modules

Workaround
This chapter describes one possible workaround. It is the decision of the integrator to choose this workaround or devise an alternative.

The following modules invoke their APIs over core boundaries or get invoked over core boundaries but are not multi-core capable yet:

> Spi

- NvM
- CSM
- TLS

In this example, the affected API calls are handled as remote procedure calls: The call to a module on a different core is redirected to a wrapper (receiver) which notifies the associated wrapper (worker) on the target core. This wrapper takes the function arguments from the receiver and calls the target API.



Caution
Reference parameter might point to a memory on which the worker has no access (e.g. stack). In such cases either the MPU setting needs to be adjusted, or the parameter copied and buffered by the receiver wrapper.

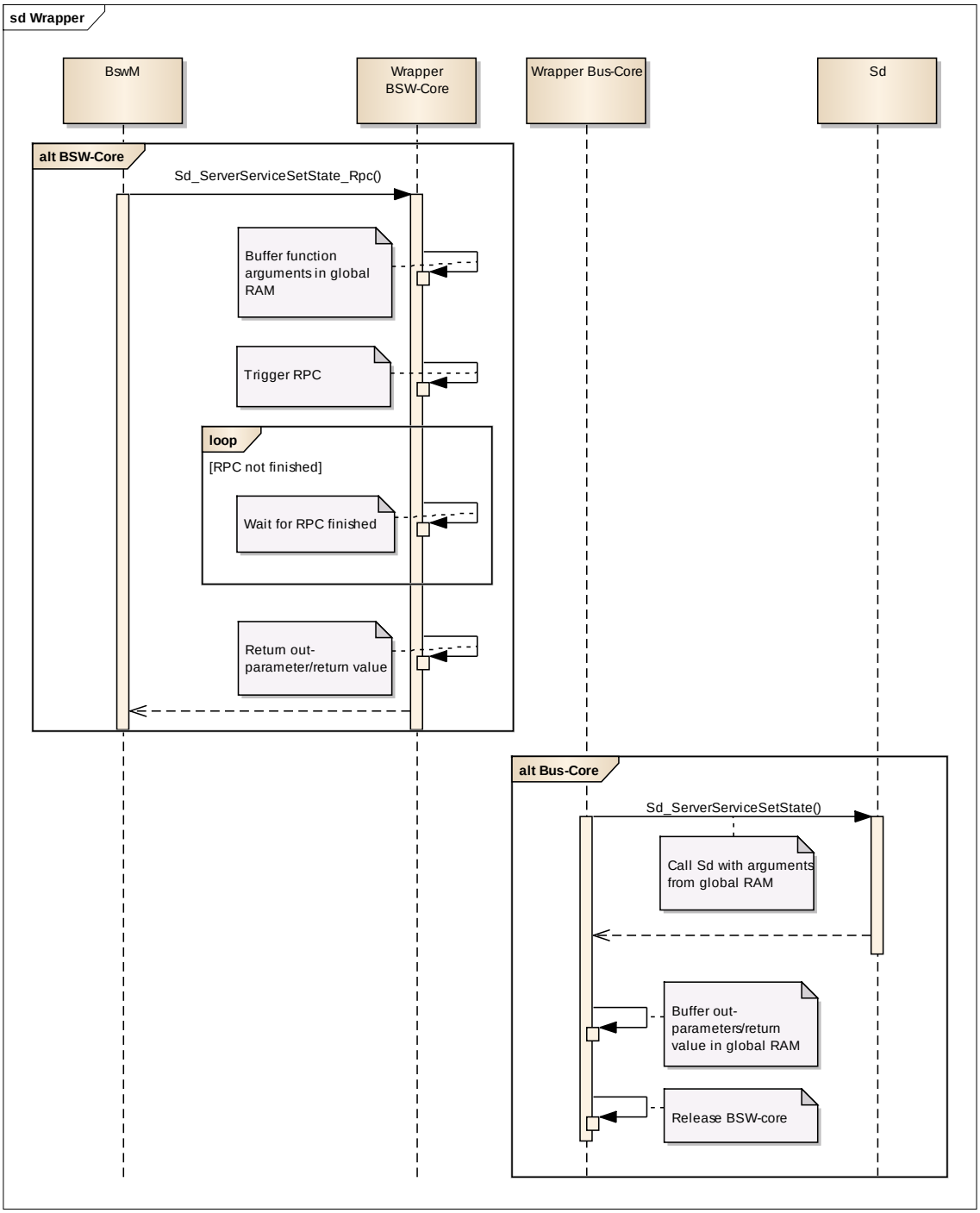


Figure 6-12 BSW-Split – Example RPC Wrapper

In this example, the target core is notified via an Os Ioc. The worker is executed in the context of the Ioc Receiver Pull Callback function. Other methods like Os Events, user implemented traps etc. are of course also valid options.

If there are no out parameter and return values, the caller core must not necessarily wait while the worker wrapper component is being executed. However, if it does not wait, the system timing behavior is different, because the remote function might still be waiting to be executed although the caller core has already finished its call.

6.3.14 Initialization

The BSW initialization is usually done by EcuM and the BswM. However, some initialization steps might need to be done in a different context by integration code. Integration code for module initialization can be executed at three different stages:

1. main() context, right after the cores startup-code
2. EcuM init lists zero and one
3. Startup task context



Note
The initialization of some modules on other partitions than the BSW partition must be configured manually in the BswM's comfort or basic editor. Please refer to chapter 3.5.

6.3.15 PduR Appl Spinlocks

The PduR uses spinlocks to protect its buffers and the following routing paths:

- TP
- Buffered
- Trigger transmit

However, since spinlocks are acquired nested and in the context of locked interrupts, OS spinlocks cannot be used. Hence a user implemented version is required.

The following APIs must be provided:

- `Std_ReturnType Appl_TryToGetSpinlock(uint32* lockVar)`
- `Std_ReturnType Appl_ReleaseSpinlock(uint32* lockVar)`
- `uint32 Appl_GetSpinlockInitVal(void)`

`Appl_TryToGetSpinlock` just tries to acquire the lock but returns immediately. Furthermore, the status of the operation is returned:

- E_OK – Lock was acquired
- E_NOT_OK – Lock was already occupied

The sequence diagram for `Appl_TryToGetSpinlock` is:

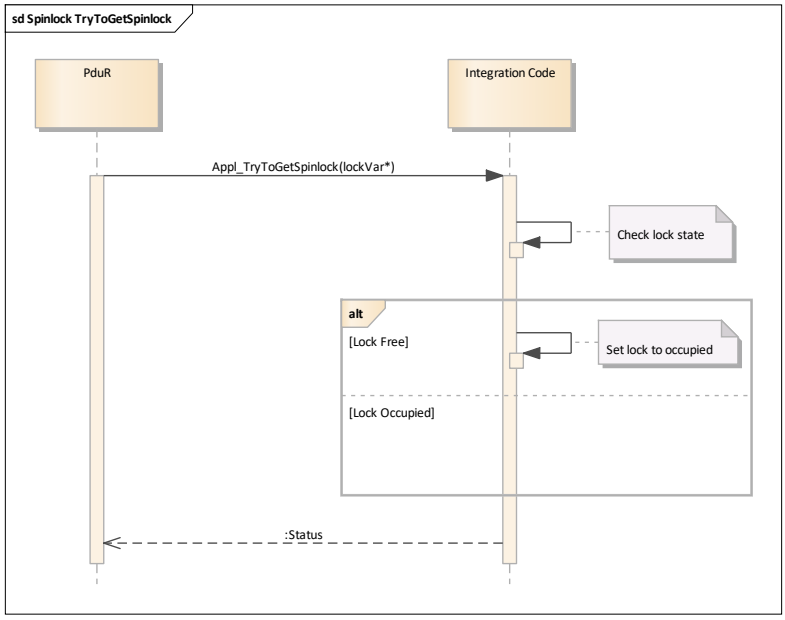


Table 6-1 BSW-Split – Example PduR Appl_TryToGetSpinlock

Note
The actual atomic acquisition of the lock can, on most µC platforms, be achieved by a Compare & Swap operation. The compilers usually provide an easy to use intrinsic to use this operation.

7 BSW-Split: Addon Multi-Instance ETH/CAN

The addons Multi-Instance ETH/CAN allow mapping of individual VLANs (IDs) respectively CAN channels to partitions with the goal of CPU load balancing. Multi-instance means, that components of the communication cluster are executed in multiple partitions and potentially at the same time. All relevant components have been extended so that transmission and reception of the sub-set of their elements (PDUs, datagrams etc.) is done within one partition only.

To fully make use of this feature, least possible overhead must be introduced. This, of course, also includes the overhead introduced by cross-partition communication of signals/PDUs. Hence, in a performance optimal configuration:

- The PDUs that are received and transmitted are processed by a Com/LdCom instance in the **same partition**
- The signals of a PDU are received and transmitted by SWC in the **same partition**

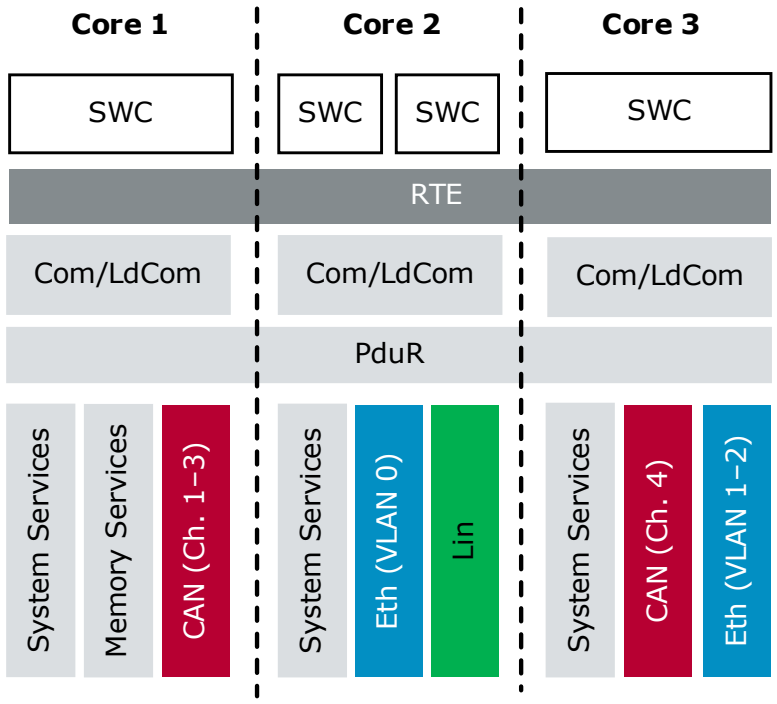


Figure 7-1 Add-On Multi-Instance ETH/CAN

Note
Multi-Instance CAN/ETH are **addon** licenses to BSW-Split.

7.1 Architecture: Multi-Instance ETH

This feature is realized by extensions of the following modules:

- Eth
- EthIf
- TcpIp
- SoAd
- PduR
- BswM
- ComM
- Nm

Each of those modules is partition aware and processes the transmission and reception of one or multiple VLANs. The PduR is the central cross-partition communication module and routes the PDUs from and to the designated upper layer modules (Com, LdCom, IpduM).

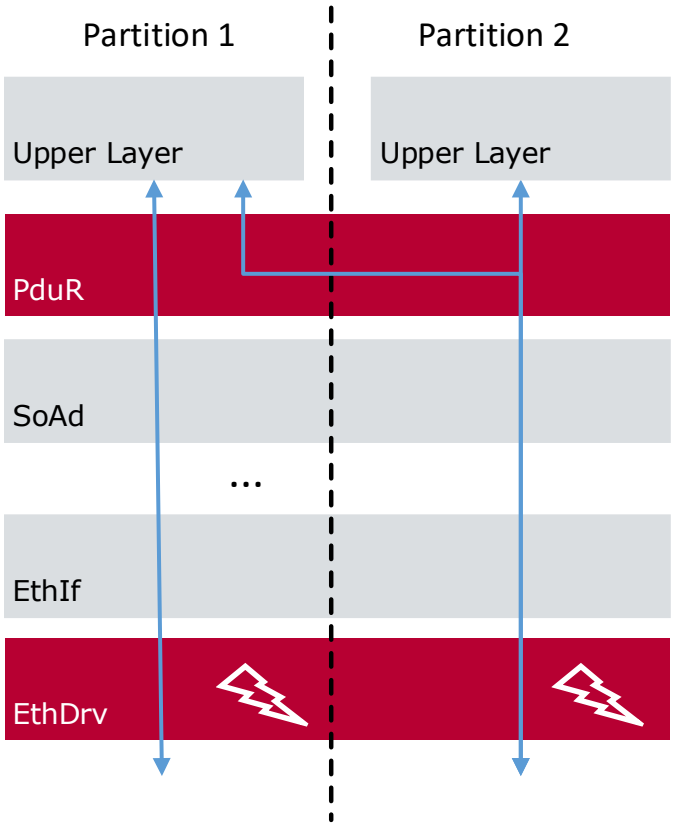


Figure 7-2 BSW-Split Multi-Instance Architecture ETH

Note
Mapping of VLANs to partitions with different ASIL is currently not supported.

7.2 Architecture: Multi-Instance CAN

This feature is realized by extensions of the following modules:

- Can
- CanIf
- CanTp
- CanTSyn
- PduR
- BswM
- ComM
- Nm

Each of those modules is partition aware and processes the transmission and reception of one or multiple CAN channels. The PduR is the central cross-partition communication module and routes the PDUs from and to the designated upper layer modules (Com, LdCom, IpduM).

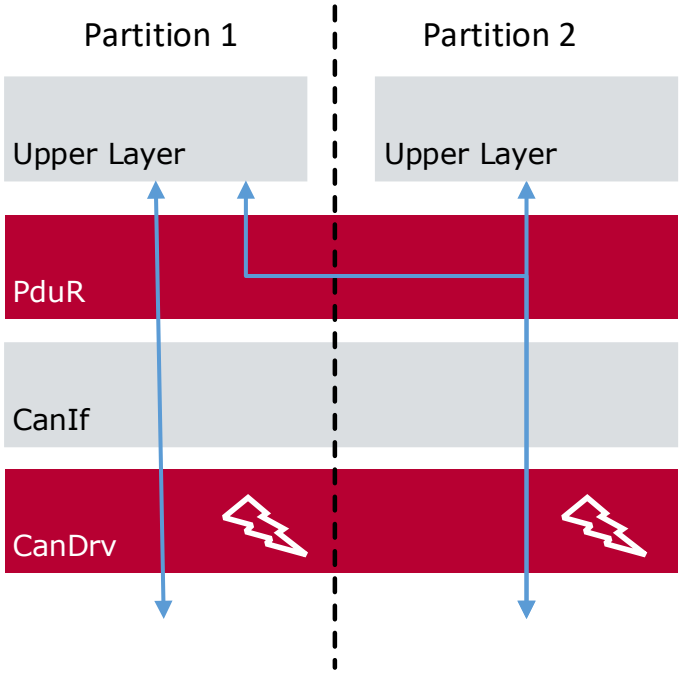


Figure 7-3 BSW-Split Multi-Instance Architecture CAN



Note
Mapping of CAN channels to partitions with different ASIL is currently not supported.

7.3 Configuration

This chapter describes the configuration steps that are required to extend a BSW-Split configuration (compare to chapter 6.3) to multi-instance.



Caution
Only strictly vertical communication paths through the CAN/ETH modules are supported. This means that the configuration elements that are mapped to partitions must be mapped such that for one CAN channel/VLAN all elements are mapped to the same ECUC partition.

7.3.1 ComM

The <bus>SM are not multi-instance capable and thus, wrapper components are required. To simplify the implementation of the wrappers as much as possible:

> Revert ComM to a single-core configuration

> Map ComM to the same partition as NmIf

7.3.2 Eth

VLAN based queueing and partition mapping of VLANs must be configured.

> Enable: `EthCtrlEnableVlanIDBasedFramesQueuing`

> Create the Rx and Tx queues (`EthRxQueue`, `EthTxQueue`)

> Assign the VLANs to queues as needed for the project:

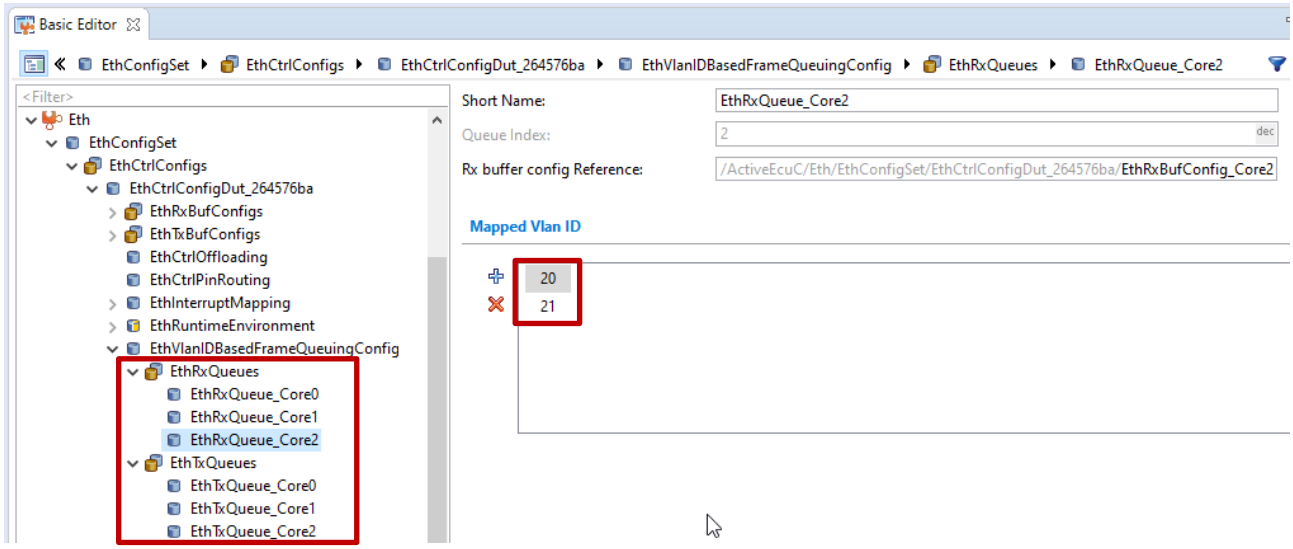


Figure 7-4 Multi-Instance ETH/CAN: VLAN to Queue Assignment

> Reference the queues for untagged frames: `EthUntaggedRxFramesQueue`, `EthUntaggedTxFramesQueue`

7.3.3 EthIf

In the Ethernet Interface, the `EthIfControllers`, as representatives of a VLAN, must be mapped to partitions.

Note
The `EtIfController` must be mapped to the same ECUC partition as the interrupt service routine (ISR) of the queue used by the corresponding VLAN.

> Configure the `EthIfEcucPartitionRef` parameters to assign VLAN IDs to a partition

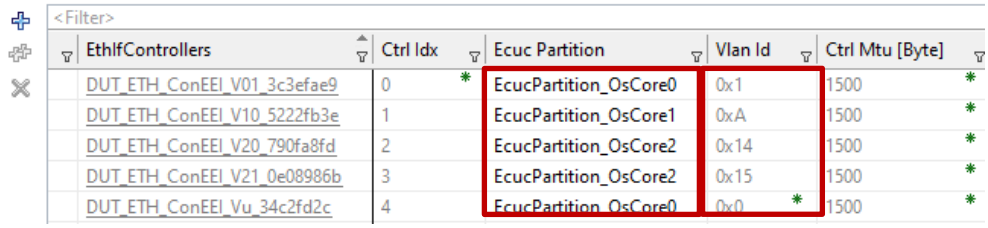


Figure 7-5 Multi-Instance ETH/CAN: EtIf Controller Mapping

7.3.4 TcpIp

In TcpIp, the TcpIp controllers (`TcpIpCtrl`) must be mapped to partitions.

> Create a `TcpIpEcuMultiPartition` container for every partition that handles VLANs

> Reference the correct partition in `EcuPartitionRef`.

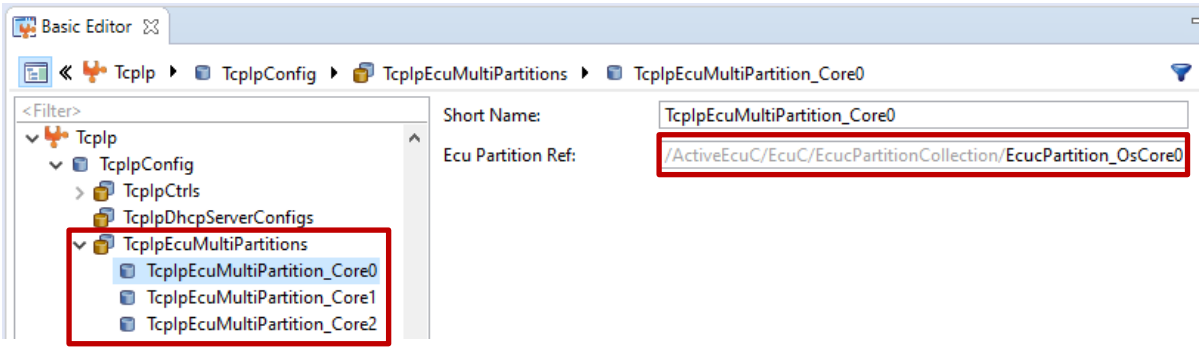


Figure 7-6 Multi-Instance ETH/CAN: TcpIp Multi Partition Configuration

> Activate multi-instance support by enabling `TcpIpMultiCoreSupportEnabled`

> Configure a Spinlock for TcpIp's Exclusive Area as described in chapter 5.20.1

7.3.5 SoAd

In the SoAd, instances must be created, that handle the Socket Connection Groups of the different partitions.

> Create a `SoAdInstance` container for every partition that handles VLANs

> Map the instances to partitions by referencing the correct partition in `SoAdPartitionRef`

> Reference the Socket Connection Groups (`SoAdSocketConnectionGroupRef`) that shall be handled by the respective instance

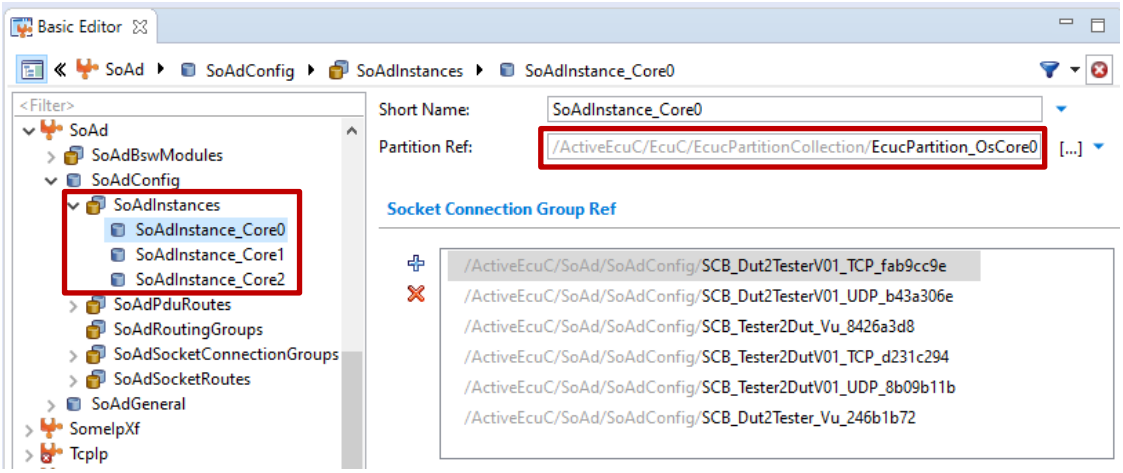


Figure 7-7 Multi-Instance ETH/CAN: SoAd Instances Configuration

> Use the solving action to map all SoAd PDUs to the correct partition in EcuC

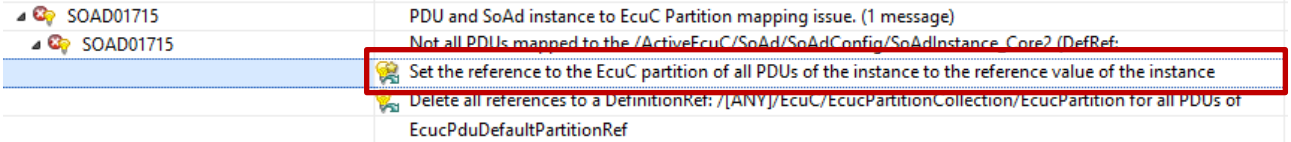


Figure 7-8 Multi-Instance ETH/CAN: PDU Partition Mapping

> Configure a Spinlock for SoAd's Exclusive Area as described in chapter 5.19.1

7.3.6 PduR

The PduR routes PDUs between upper- and lower layers on the same- and between different partitions. In most cases, there are different EcuC global PDUs for source- and the destination of the PduR Routing Path:

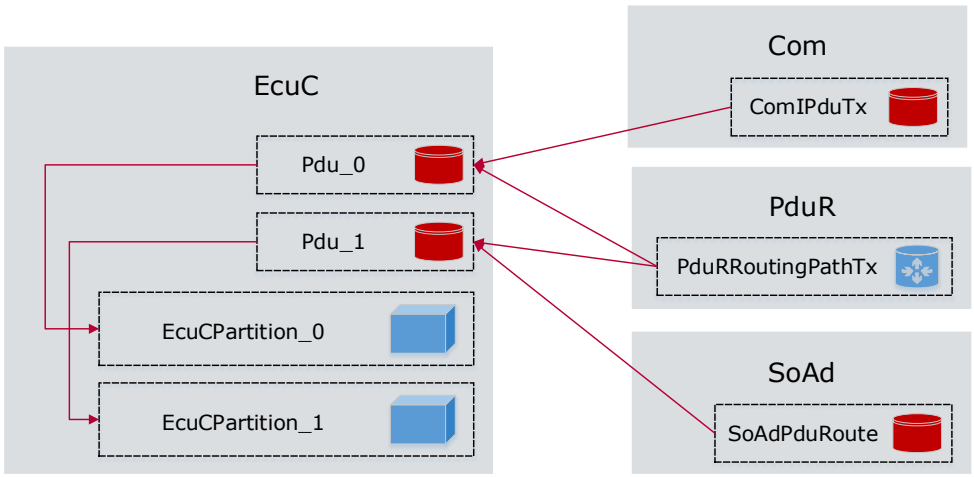


Figure 7-9 Multi-Instance ETH/CAN: Individual PDUs for Routing Path

In case the PDUs reference the same partition, the PduR creates a single-core routing path. If, however, the two PDUs refer different partitions, the PduR will create a multi-core routing path.

In both cases, the partition assignment is expressed by the `EcucPduDefaultPartitionRef`.

If, on the other hand the routing path uses the same EcuC global PDU for source- and destination on multi-partition routings, the `EcucPduDedicatedPartition` must be used.

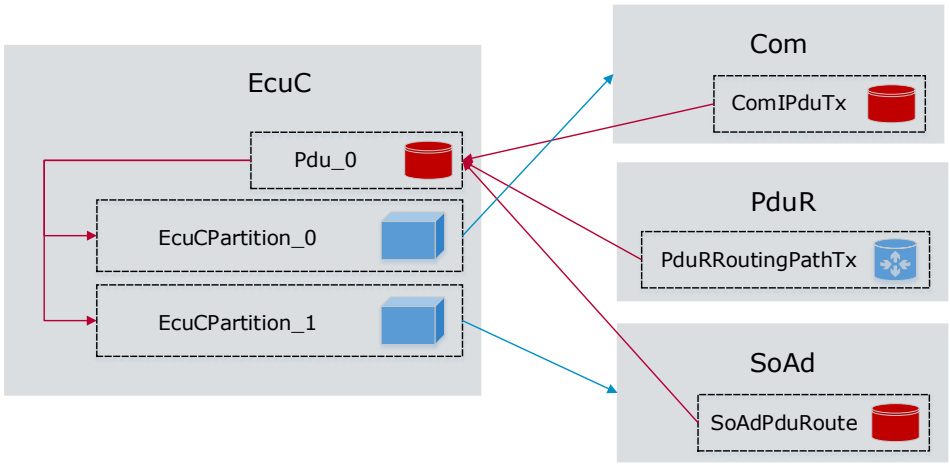


Figure 7-10 Multi-Instance ETH/CAN: Shared PDUs for Routing Path with Dedicated Partition Ref

7.3.7 NmIf

The <bus>Nm are not multi-instance capable. Thus, wrapper components are required. To simplify that as much as possible

> Revert NmIf to a single-core configuration

7.3.8 BswM

The BswM configuration must be extended to

> Initialize the BSW modules on the partitions that handle VLANs/CAN channels

> Perform Communication control as described in chapter 5.1

7.3.9 Can

In Can, multi-instance support must be enabled:

> Activate parameter `CanSupportMultipartition`

7.3.10 CanIf

Adapt the CanIf-to-partition assignment and ECUC PDU partition mapping so that the CanIf Controllers are mapped to different partitions. How the configuration is done in general is described in chapter 6.3.4.

7.3.11 CanTp

In CanTp, the TP SDUs must be mapped to partitions.

> Activate the multi-instance feature by enabling the parameter `CanTpSupportMultiPartition`

> Map the CanTp SDUs (Rx, Tx) to the partitions on which the CAN PDUs are received/transmitted by referencing the partition in `CanTpRxNSduPartitionRef` / `CanTpTxNSduPartitionRef`

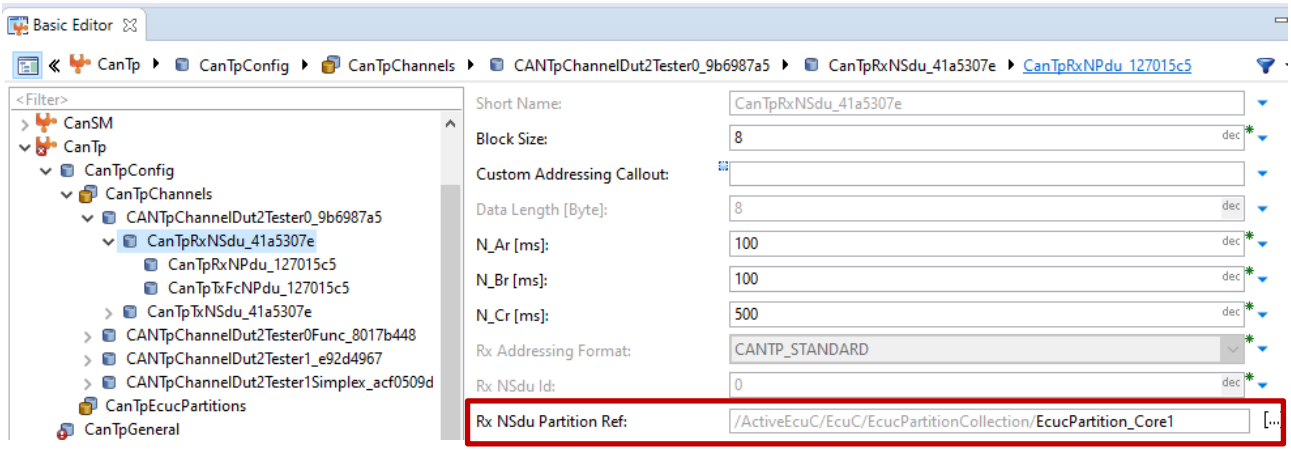


Figure 7-11 Multi-Instance ETH/CAN: CanTp SDU Mapping

7.3.12 OS

The OS configuration may need to be extended with appropriate tasks for the partition specific main functions.

Furthermore, the CAN/Eth ISR must be mapped to the correct OS Applications.

Note
The partition/OS Application of the **Can ISR** can be determined in CanIf by following the reference: `CanIfCtrlCanCtrlRef`.

Note
The partition/OS Application of the **Eth ISR** can be determined via multiple hops:

The name of the Eth ISR is created with the following schema: `EthIsr_EthCtrlConfigDut_264576ba_EthInterruptServiceRoutine_Q`**<EthQueueIndex>[Rx|Tx]**.

The connecting element between queue (→ ISR) and partition is the VLAN ID. Every VLAN is handled by a `EthIfController` that finally references a partition (`EthIfEcucPartitionRef`)

7.3.13 Runtime System

In the Runtime System section, all main functions must be mapped to tasks in the corresponding partitions.

The names of the multi-instance specific main functions are decorated with the OS Application, they need to go to:

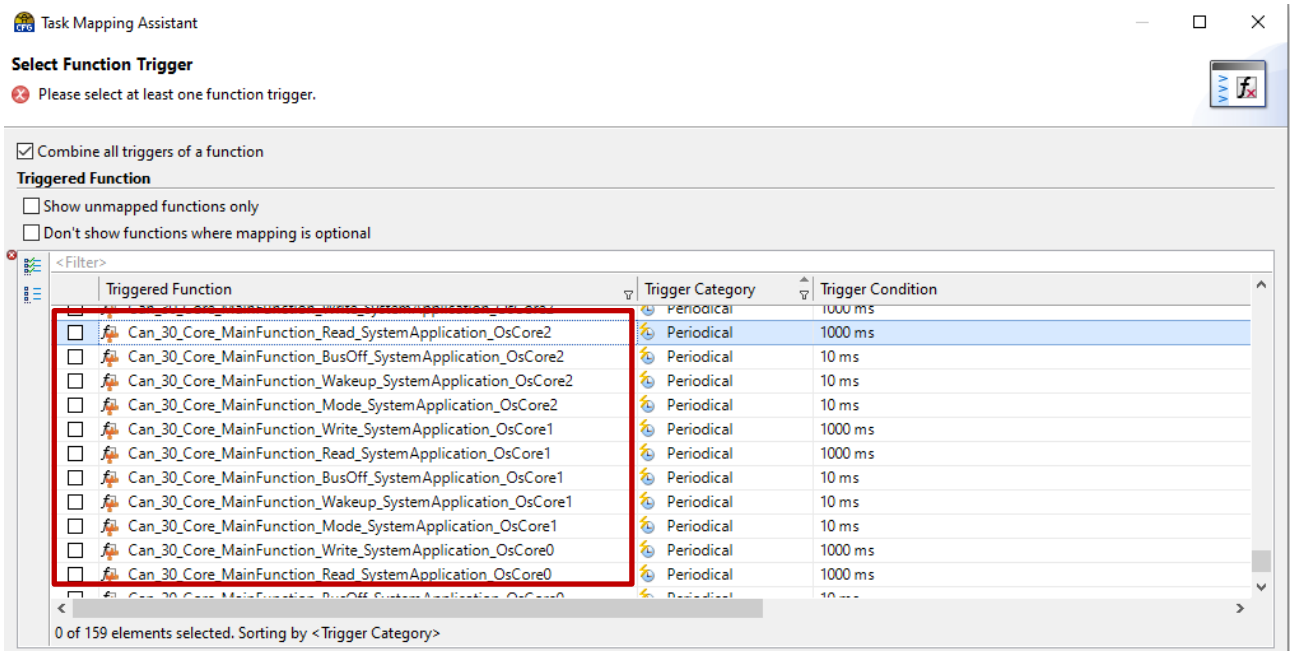


Figure 7-12 Multi-Instance ETH/CAN: Main Function Mapping

7.4 Wrapper Components

State- and Network management is currently not multi-instance capable. That is why, wrapper-components are required. The general idea is described in chapter 6.3.13.

In a multi-partition configuration, the wrapper must determine to which partition the addressed instance is mapped to.

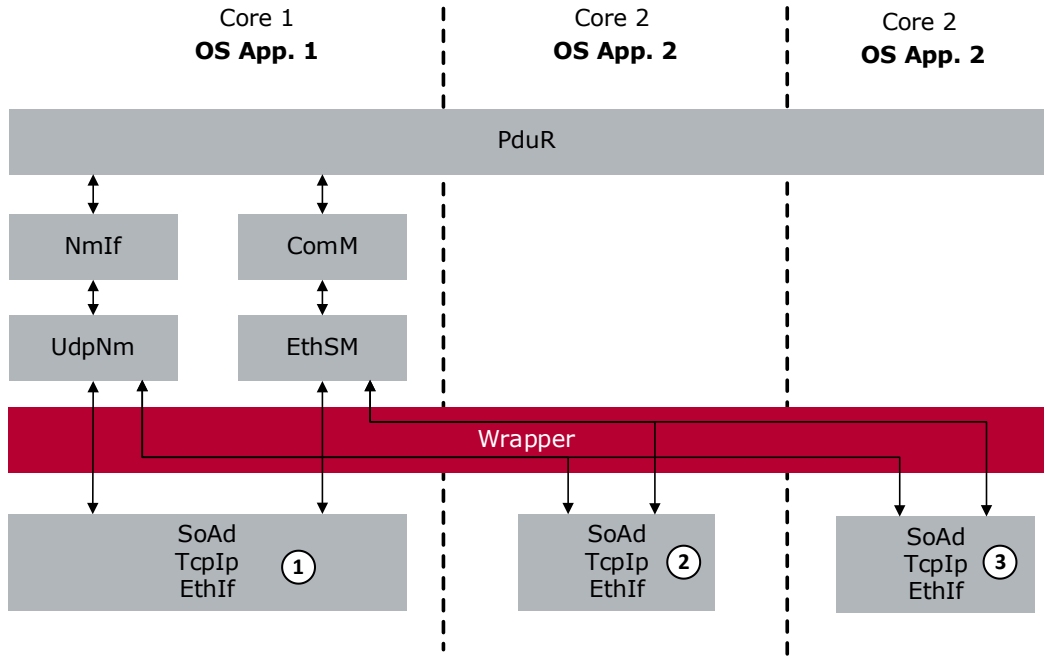


Figure 7-13 Multi-Instance ETH/CAN: Wrapper Architecture

The example in Figure 7-13 shows an example configuration:

- UdpNm and EthSM are on the same partition as SoAd, TcpIp and EthIf #1. The wrapper can thus simply forward the called API directly
- SoAd, TcpIp and EthIf #2 and #3 are on different partitions. The wrapper must hence forward the API calls to the correct partition as described in chapter 6.3.13

Figure 7-14 shows the pseudo-code of an example API. For this API, the parameter `ControllerId` is used to determine the partition assignment of the correct CanIf instance. For other APIs different parameters must be identified, by which the partition mapping can be determined.

```
Std_ReturnType CanIf_SetControllerMode_Wrap(uint8 ControllerId, CanIf_ControllerModeType ControllerMode)
{
   switch ControllerId

       case CONTROLLER_ON_PARTITION_1:
           // CanIf instance is on same partition
           CanIf_SetControllerMode(ControllerId, ControllerMode)

       case CONTROLLER_ON_PARTITION_2
           // CanIf instance is on another partition (-> Partition 2)
           ForwardCall(PARTITION_2, ControllerId, ControllerMode)

       case CONTROLLER_ON_PARTITION_3
           // CanIf instance is on another partition (-> Partition 3)
           ForwardCall(PARTITION_3, ControllerId, ControllerMode)

}
```

Figure 7-14 Multi-Instance ETH/CAN: Target Instance Determination



Caution
Parameters change with the configuration. It must be verified, that the values used in the wrapper code to determine the partition mapping are still correct after configuration changes.

7.4.1 Affected APIs

This chapter contains a list of APIs that potentially must be handled by a wrapper. If- or if not, an API needs to be handled depends on the configuration.

Origin: CanSM

```
CanIf_CheckTrcvWakeFlag
CanIf_ClearTrcvWufFlag
CanIf_GetPduMode
CanIf_GetTxConfirmationState
CanIf_SetControllerMode
CanIf_SetPduMode
CanIf_SetTrcvMode
CanIf_SetBaudrate
```

Origin: CanIf

```
CanNm_RxIndication
CanNm_TxConfirmation
CanSM_CheckTransceiverWakeFlagIndication
CanSM_ClearTrcvWufFlagIndication
CanSM_ConfirmPnAvailability
CanSM_ControllerBusOff
CanSM_ControllerModeIndication
CanSM_TransceiverModeIndication
CanSM_ControllerModeIndication
```

Origin: EthSM

```
EthIf_SetControllerMode
EthIf_GetControllerMode
TcpIp_RequestComMode
```

Origin: EthIf

```
EthSM_CtrlModeIndication
EthSM_TrcvLinkStateChg
```

Origin: TcpIp
`EthSM_TcpIpModeIndication`

Origin: UdpNm
`SoAd_IfTransmit`

7.5 Startup-Synchronization

Note
Applies only to configurations that use the Gpt as time source in StbM

There are the following boundary-conditions when using StbM and Gpt in a multi-partition configuration.

- Gpt must be initialized on the master partition first (depending on the concrete driver: Check the technical reference)
- Gpt must be initialized on the slave partitions after the initialization of the master.
- Since the StbM uses the Gpt, it must be initialized after Gpt
- The function `StbM_Init()` must be called on all partitions that handle a StbM instance
- In a final step, the API `StbM_PostInit()` must be called after initialization of all StbM instances.

To enforce this temporal order, the cores must be synchronized. This can be done with OS Barriers.

There are two barriers needed:

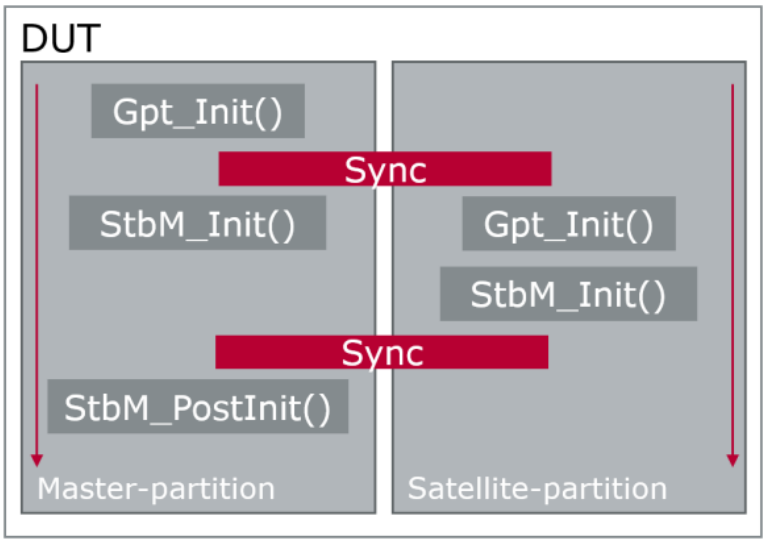


Figure 7-15 Multi-Instance ETH/CAN: Startup Synchronization with OS Barriers

Configuration- and invocation of the barriers (`Os_BarrierSynchronize()`) must be done in integration code developed by the user.

8 Application SWC Design Recommendations

This chapter gives a collection of basic architectural recommendations and general hints for the development of efficient multi-core application software. However, since software requirements differ vastly, there is no universal approach which can be applied to all projects. Furthermore, the behavior of the BSW and Rte depends on a huge variety of configuration parameters and application design aspects. Giving a complete description would easily exceed the scope of this document.

For this reason, this chapter is meant to be a pool of general guidelines from which the most applicable- and suitable items can be picked during development.

8.1 Code Design

This chapter contains the guidelines for designing the software functionality.

- It is not possible to map runnables of a SWC to different cores. Hence, an SWC should be designed to bundle only those runnables which are going to be mapped to the same core.
- Bundle tightly coupled collections of SWCs in compositions. Hence relocating to other cores in the scope of load balancing can be achieved simpler.
- Favor cyclic IPO (Input-Process-Output) design patterns (e.g. Simulink models). IPO supports simple distribution of the different steps to different cores. Furthermore, cyclic execution and hence communication, can be realized with less overhead by the Rte than event driven communication.

8.2 Inter-Core Data Flow

Inter-core data flow is always expensive in terms of overhead runtime and should therefore be reduced to an absolute minimum. Depending on the type of data flow (e.g. between SWC or between SWC and BSW) the communication path is more or less complex and hence runtime intensive.

The guidelines stated in the following chapter focus on keeping the overhead runtime to a minimum:

- Minimize access from a remote core to COM signals to an absolute minimum. Crossing the core-boundary to COM is runtime intensive. Especially for write accesses.
- Make use of simple, atomically read- and writable data types in Send/Receive (S/R) port interfaces instead of complex types. Port interfaces with native data types are realized as simple variables in shared memory with no additional protection whatsoever whereas complex data types require Spinlocks.
  - Deduction 1: Disaggregate complex data types of S/R port interfaces and use several native data type elements instead. This however is only sensible if data consistency is either less critical or still given by other mechanisms such as scheduling sequences.
  - Deduction 2: If complex data types are crucial for securing data consistency, aggregate data elements of several S/R port interfaces which can be read- or written

simultaneously. Thus, only a single large data element has to be transferred via IOC instead of several smaller elements. This results in a better overhead to payload ratio in the IOC communication.

- Prefer un-queued (Last-Is-Best) communication in S/R port interfaces (Read Data) instead of queued communication (Receive Data). The queue-mechanism needs to be protected from concurrent access which is done by Spinlocks.
- Use buffered port access for S/R Port Interfaces (Implicit Port access). Hence port data is buffered in a runnable-exclusive, core-local temporary memory. For receive Port Interfaces, the data is fetched and buffered immediately before the runnable is called while data transmitted via Send Port Interfaces is written back not until the runnable has finished. As result the runtime-costly crossing of core-boundaries is needed only once for each Port Interface. No matter how often the Port Interface is actually accessed by the Runnable's application code. Drawback is of course that more RAM is needed.
- Schedule runnables which cyclically access S/R Port Interfaces with cycle times as long as tolerable. The result is less IOC-usage per given period of time and ultimately less overhead runtime.
- Avoid runnable activations triggered by data related events of S/R Port Interfaces like:
  - On Data Reception
  - On Data Reception Error
  - On Data Send Completion

  These require OS mechanisms (events, task activations) on remote cores to schedule runnables which adds runtime overhead but it may reduce spinning time.

8.3 Software Distribution/Mapping

This chapter describes approaches for mapping respectively distributing the runnables/SWC to cores.

- Map all SWCs of a single ECU-function to tasks on the same core. This follows the assumption that coupling between ECU-functions is rather low and requires fewer Port Interfaces across core boundaries than an approach in which the SWC of a function are distributed.
- Map SWC which make heavy use of BSW-services (e.g. communication) to tasks bound to the BSW-core. This eliminates the need to cross core boundaries via RPC for each call to the BSW. Especially for frequently used interfaces, even small overheads can sum up considerably.
- Map computational intensive algorithms/SWC to tasks to a remote-core. This is especially useful if CPU-load of the BSW-core shall be reduced. This has a positive effect as soon as the runtime of the relocated SWC is greater than the overhead of the core-crossing of its Port Interfaces.
- Map software parts with chained computation order to the same core. This especially applies to configurations in which the order is defined by the mapping of Runnables to

tasks. Simple sequences on the same core- and task with the same cyclic timing can be realized very efficiently as a sequence of function calls. Distributed over cores however, OS-mechanisms are required to ensure the proper execution order.

> Map entire Simulink models to remote cores. Such models with their comparably long runtime are ideally suited for multi-core especially when all input data is read once at the beginning of the computation step and written back once after its completion.

8.4 Configuration Aspects

Avoid non-preemptive tasks. Especially when (time critical) execution-sequences are distributed over several cores. Otherwise, due to the Os scheduling policy, a high priority task may have to wait for a low priority, non-preemptive task to finish on a remote core. On the other hand, a non-preemptive task may require more effort to handle data consistency issues.

9 Glossary and Abbreviations

9.1 Glossary

Term	Description
EAD	Embedded Architecture Designer; generation tool for MICROSAR components
GENy	Generation tool for CANbedded and MICROSAR components

Table 9-1 Glossary

9.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
BswM	BSW Mode Manager
CAN	The CAN cluster
Can	CAN driver
CanIf	CAN Interface
CanTP	CAN Transport Layer
CanTSyn	Time Sync Over CAN
Com	Communication
ComM	Communication Manager
Dem	Diagnostic Event Manager
Dem	Diagnostic Event Manager
Det	Development Error Tracer
E2E	End to end protection
EAD	Embedded Architecture Designer
ECU	Electronic Control Unit
EcuM	ECU State Manager
ETH	The ethernet cluster
Eth	Ethernet driver
EthIf	Ethernet Interface
FiM	Function Inhibition Manager
HIS	Hersteller Initiative Software
IpduM	I-PDU Multiplexer
ISR	Interrupt Service Routine
LdCom	Large Data COM
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
NmIf	Network Management Interface

Os	Operating system
PduR	PDU Router
RPC	Remote Procedure Call
PPORT	Provide Port
RPORT	Require Port
Rte	Runtime Environment
SIL	(Atomotive) Software Integrity Level
SRS	Software Requirement Specification
StbM	Synchronized Time-Base manager
SoAd	Socket Adaptor
TcpIp	TCP/IP Stack
SWC	Software Component
SWS	Software Specification
WdgIf	Watchdog Manager
veRtm	Runtime Measurement
Xcp	XCP (Universal Calibration Protocol)

Table 9-2 Abbreviations

10 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com